



RadonPy



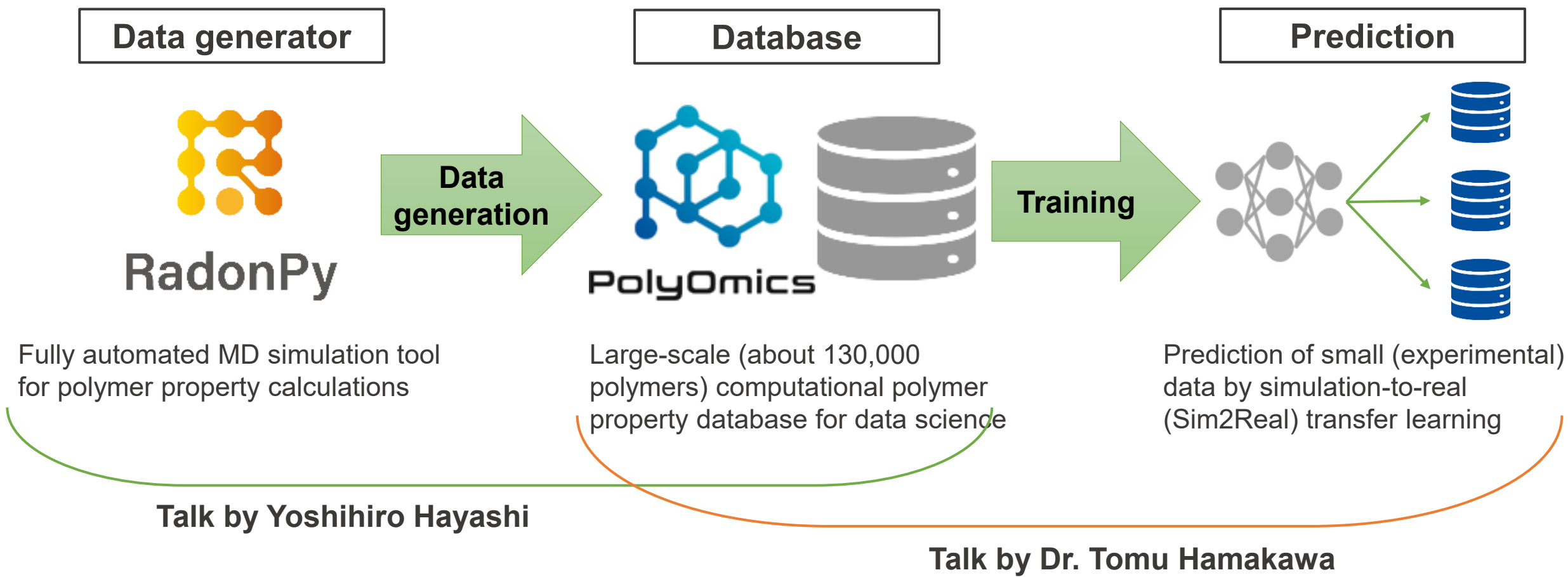
PolyOmics

RadonPy Tutorial: From Automated Molecular Dynamics Simulations to Real-World AI Applications

Yoshihiro Hayashi, Tomu Hamakawa
The Institute of Statistical Mathematics
yhayashi@ism.ac.jp

2nd July 2026
CECAM-DE-SMSM

Overview of our tutorial



Concept of RadonPy

Motivation of development of RadonPy

In machine learning, prediction accuracy improves when the training data size increases.

How can we obtain a large-scale dataset in polymer material science?

RadonPy is a data generator in the AI era rather than a simulation tool for physical chemistry.

- ✓ Top priority of RadonPy is high-throughput, not high-accuracy.
 - The calculated values are calibrated by machine learning. Prediction accuracy of machine learning is more important.
- ✓ The commoditization of polymer MD simulations
 - Implement “preset modules” which enable non-experts to easily perform the appropriate MD simulation.
- ✓ Python-based and open source to allow for flexible integration into systems
 - Easy to integrate with Python-based library of data science.

Software overview of RadonPy



Y. Hayashi et al. "RadonPy: Automated Physical Property Calculation using All-atom Classical Molecular Dynamics Simulations for Polymer Informatics," *npj Computational Materials*, **8**, 222 (2022)

Source code: <https://github.com/RadonPy/RadonPy>

RadonPy

Fully automated MD simulations for polymer property calculation

- It can implement automated pipelines including initial structure generation, atomic charge calculation, force field assignments, equilibration MD, non-equilibrium MD.
- The latest version implements automatic MD for 61 properties.

Easy installation and high portability

- Composed of OSS only (LAMMPS, Psi4, RDKit, etc.)

Easy calculation by “preset” modules

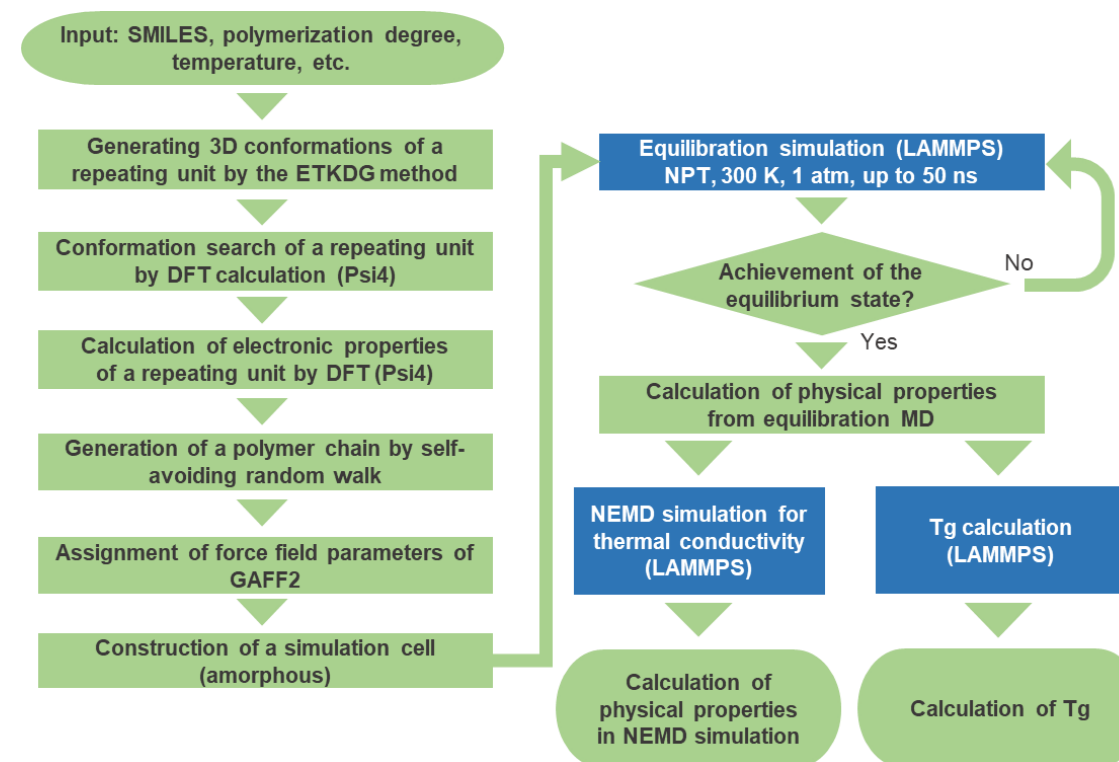
- Implementation “preset modules” to easily and appropriately calculate the physical properties of polymers
- Preset modules can be added to RadonPy via an add-on.

Open innovation by involvement of experimental and data scientists

- RadonPy was used to generate structures in OPoly26 which is dataset for machine learning potential of polymers.

Daniel S. Levine, et al., arXiv:2512.23117 (2026)

Workflow of the automatic MD using RadonPy



List of properties available in RadonPy

Physical properties

- Density
- Molecular weight
- vdW volume
- Radius of gyration (Rg)
- Scaled Rg
- End-to-end distance
- Nematic order parameter
- Self-diffusion coefficient
- Bulk modulus
- Compressibility

Thermal properties

- C_p
- C_v
- Volume expansion coefficient
- Linear expansion coefficient
- Thermal conductivity (TC) ([preset.tc](#))
- TC decomposition analysis (8 components) ([preset.tc](#))
- Thermal diffusivity ([preset.tc](#))
- Glass transition temperature (Tg) ([preset.tg](#))

Dielectric properties

- Static dielectric constant
- Corrected static dielectric constant
- Dynamic dielectric constant ([preset.ef_dp](#))
- Dielectric loss ([preset.ef_dp](#))
- Dielectric loss tangent ([preset.ef_dp](#))

Optical properties

- Refractive index
- Frequency dependent refractive index (486, 589, 656 nm) ([requiring TD-DFT](#))
- Abbe number ([requiring TD-DFT](#))

Solubility properties

- Cohesive energy density ([preset.sp](#))
- Hildebrand solubility parameter ([preset.sp](#))
- Hansen-like solubility parameters (4 components) ([preset.sp](#))

Mechanical properties

(experimental implementation)

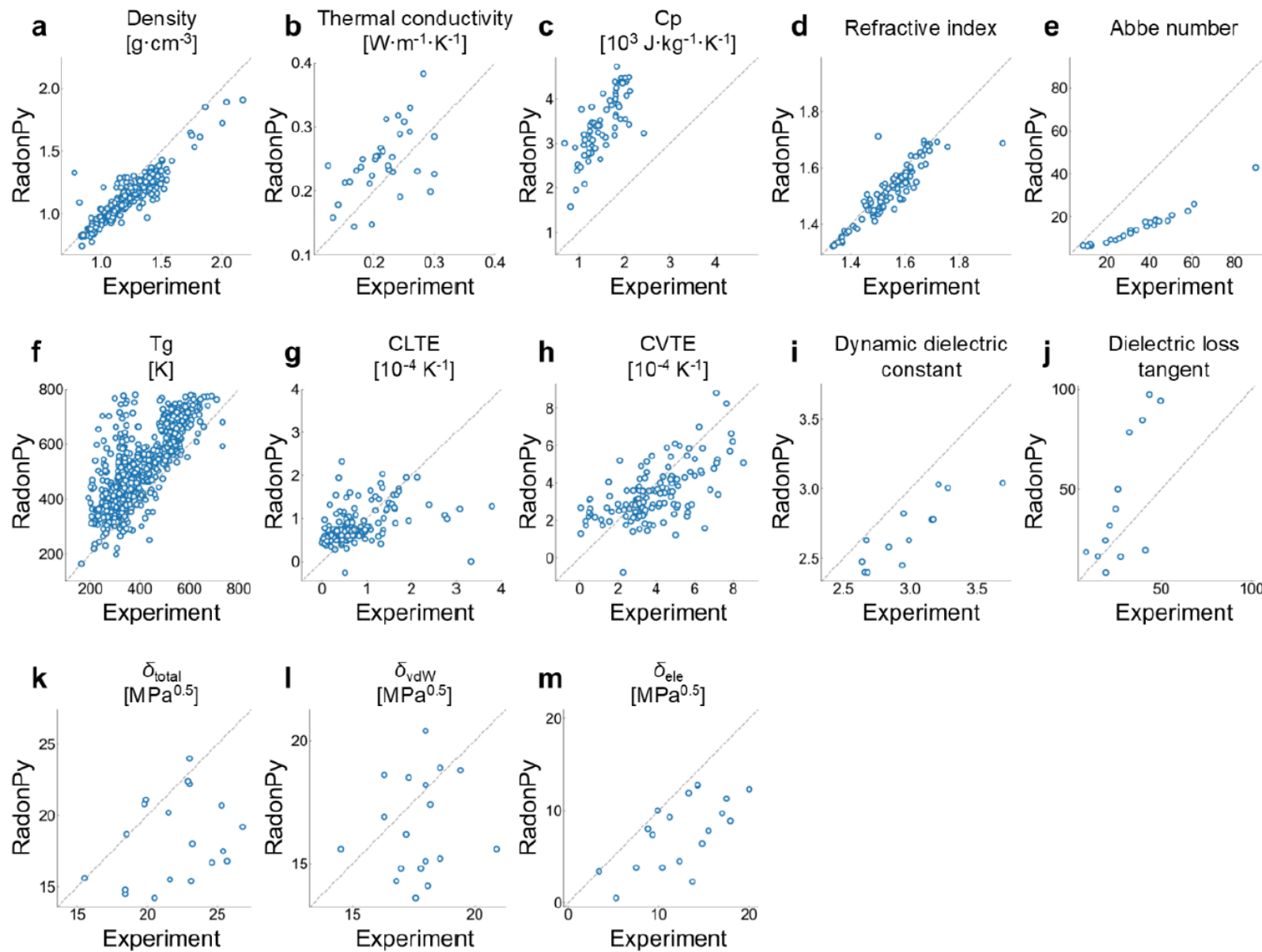
- Young's modulus ([preset.tem](#))
- Poisson's ratio ([preset.tem](#))
- Shear modulus ([preset.tem](#))
- Bulk modulus ([preset.tem](#))
- Lamé's constant ([preset.tem](#))

DFT calculated properties

(for repeating unit)

- Total energy
- HOMO
- LUMO
- Dipole moment
- Polarizability
- Dynamic polarizability (486, 589, 656 nm) ([requiring TD-DFT](#))

Gap between high-throughput simulations and experiments



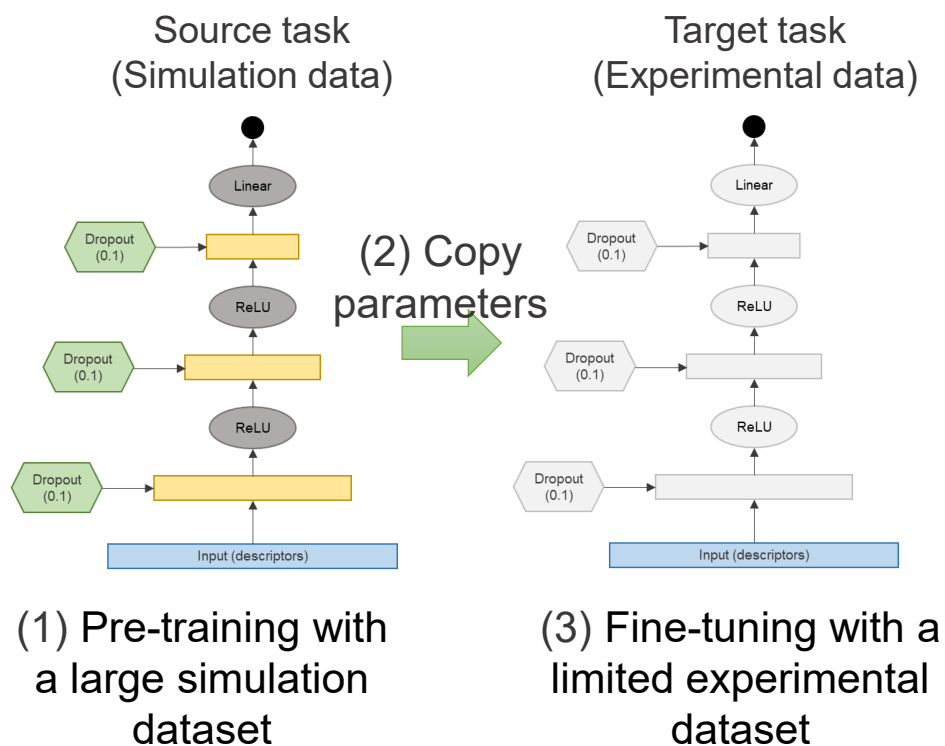
- Comparison between MD simulations and PoLyInfo experimental data
- Good agreement: density, thermal conductivity, and refractive index
- Systematic bias: C_p , Abbe number, T_g , dynamic dielectric constant, loss tangent, solubility parameters of total and electrostatic components.
- Large variance: linear and volumetric expansion coefficients, solubility parameters of vdW component
- These gaps can be reduced by **Sim2Real transfer learning**.

Simulation-to-real (Sim2Real) transfer learning

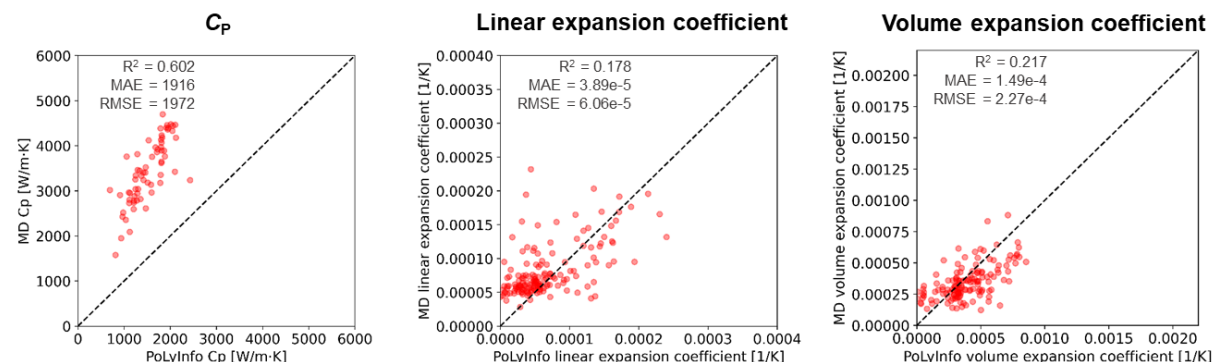
- We perform transfer learning to bridge the gap between simulation and experiments.
- The bias and variance are reduced.
- MD data are not perfect, but they are useful for transfer learning.

Transfer learning

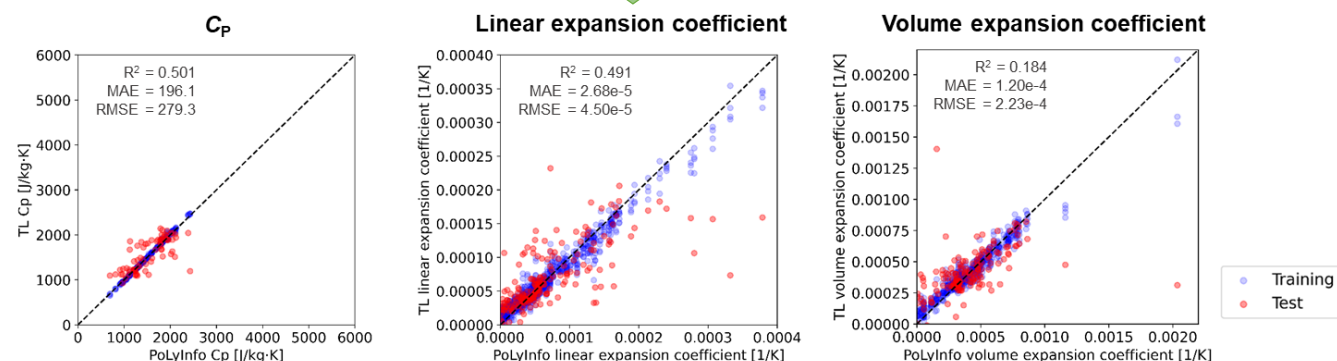
- ◆ Use knowledge learned from a large source dataset for a small target dataset.



(a) MD simulation



(b) Transfer learning



PolyOmics: Large-Scale Polymer Property Database



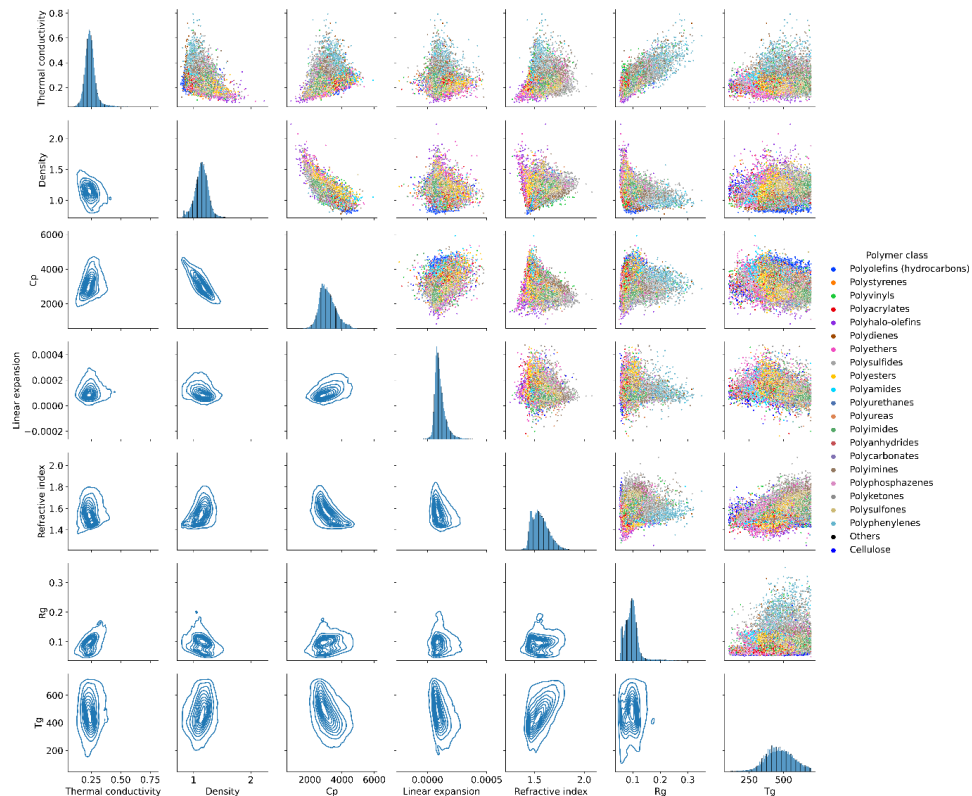
PolyOmics

- Full-scale data production on supercomputers started in January 2022.
- Calculations for approximately 130,000 polymers were completed by October 2025.
- A preprint paper has been published.

R. Yoshida, Y. Hayashi et al., *arXiv preprint*, arXiv:2511.11626 (2025)

Initial visualization of the property distribution of polymers in the 10^5 -scale database

Joint distributions of density, thermal conductivity, glass transition temperature, radius of gyration, Tg distribution, etc.



PolyOmics database

<https://huggingface.co/datasets/yhayashi1986/PolyOmics>

Description	# of polymers
Amorphous polymers	125,658
Thermal conductivity after uniaxial stretching	1,379
Dynamic dielectric properties	1,084
Abbe number / frequency-dependent refractive index	13,240
Cellulose derivatives	50,661
Small molecules	23,361
PFASs	3,821
Ladder polymers	519
Solubility parameters	58,834
ML predicted χ -parameters for 19 solvents	93,331

Installation of RadonPy via conda

1. Download of miniforge

<https://github.com/conda-forge/miniforge>

2. Installation of miniforge on Linux system

```
bash ./Miniforge3-Linux-x86_64.sh
```

3. Create conda environment (Python 3.13)

```
conda create -n radonpy python=3.13 -c conda-forge  
conda activate radonpy
```

4. Installation of requirement packages by conda

```
conda install -c conda-forge rdkit psi4 dftd3-python resp scipy psutil mdtraj matplotlib pandas pip
```

5. Installation of LAMMPS by conda

```
conda install -c conda-forge lammmps
```

or manually build from source of LAMMPS official site. The preset module of solubility parameters (preset.sp) requires to install TALLY package in LAMMPS.

In this case, the environment variable must be set:

```
export LAMMPS_EXEC=<path-to-LAMMPS-binary>
```

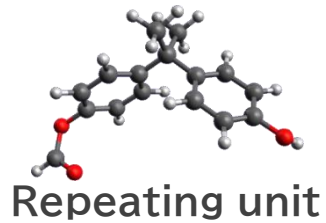
6. Installation of RadonPy 1.0 beta version

```
git clone -b main https://github.com/RadonPy/RadonPy.git  
pip install --no-index --find-links=./RadonPy/dist/ radonpy-pypi
```

Note: Runtime errors may occur in Psi4 when installed Python packages from different channels are mixed.

Example pipeline of RadonPy

Oc1ccc(cc1)C(c1ccc(cc1)OC(=O))(C)C
SMILES



Input: SMILES, polymerization degree,
temperature, etc.

Conformation search of a repeating unit
(LAMMPS & Psi4)

Calculation of electronic properties
of a repeating unit (Psi4)

Polymer chain
(about 1,000 atoms)

Generation of a polymer chain

Assignment of force field parameters

Amorphous cell
(10 polymer chains)

Generation of a simulation cell

Equilibration simulation (LAMMPS)

Reaching
equilibrium
state?

No

Yes

Calculation of physical properties in equilibration
simulation

Sim. for
thermal
conductivity

Thermal
conductivity

Sim.
for Tg

Tg

Sim.
for SP
values

SP
values

Sim. for
dynamic
dielectric
properties

Dynamic
dielectric
properties

Quick start

We prepared scripts which were used in the data generation of PolyOmics database.

- **0_qm.py** : For a repeating unit, 3D coordinates generation, conformation search, geometry optimization (DFT), atomic charge (RESP) calculation (HF), polarizability calculation (DFT).
- **1_eq.py** : Generation of polymer chains (homopolymers and copolymers), assignment of force field parameters, generation of an amorphous structure as an initial structure, equilibration MD simulation, check equilibrium state.
- **2_rst_eq.py** : Restarting equilibration MD simulation if system does not reach equilibrium state after eq.py.
- **3_tc.py** : Calculation of thermal conductivity by non-equilibrium MD (NEMD) simulation using the preset (preset.tc)
- **4_tg.py** : Calculation of T_g by temperature-cooling MD simulation using the preset (preset.tg)
- **5_sp.py** : Calculation of solubility parameters using the preset (preset.sp)
- **6_ef_dp.py** : Calculation of dynamic dielectric constants and loss tangent by NEMD simulation using preset.ef_dp

Run the scripts in the order listed above.

These scripts can be downloaded via GitHub site:

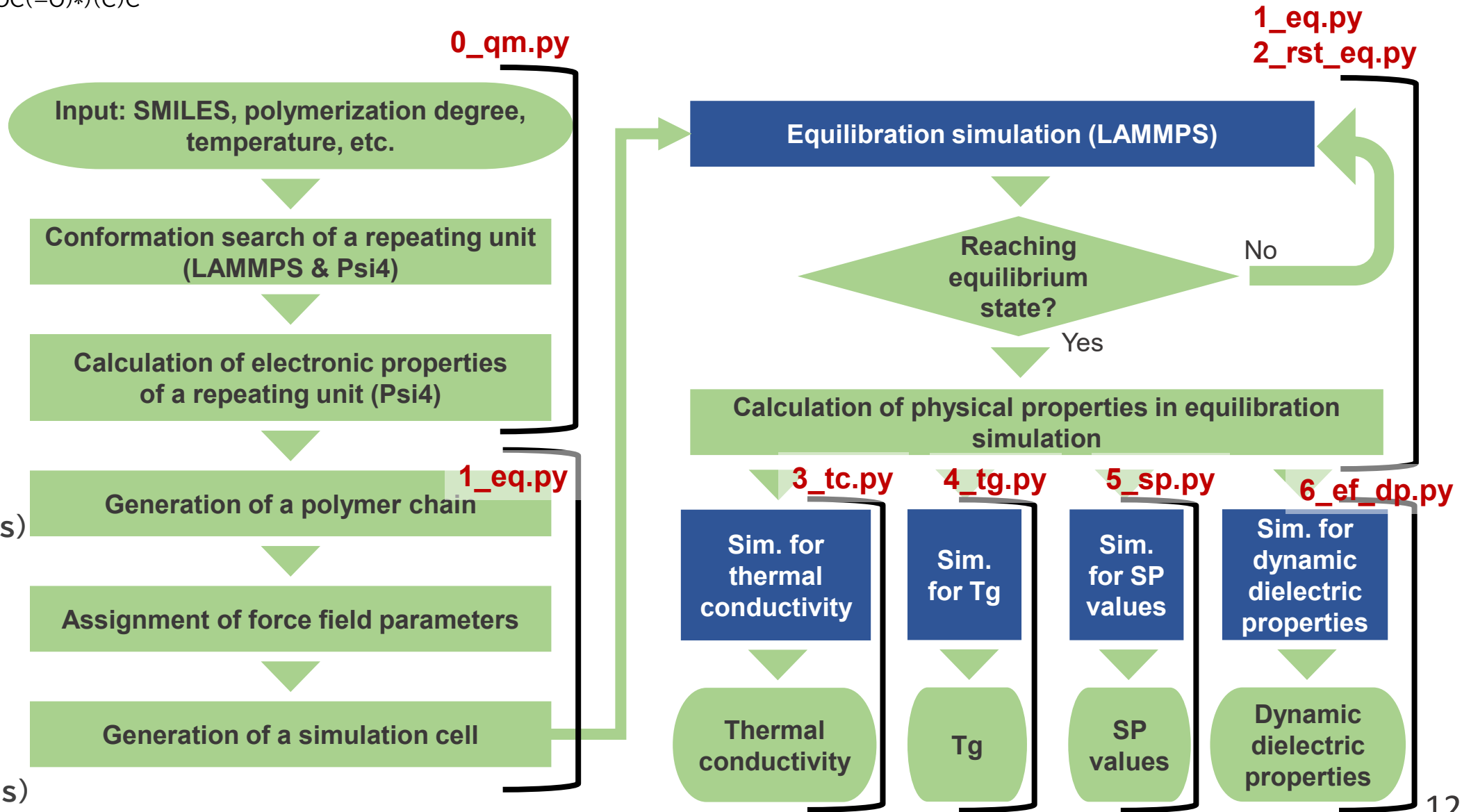
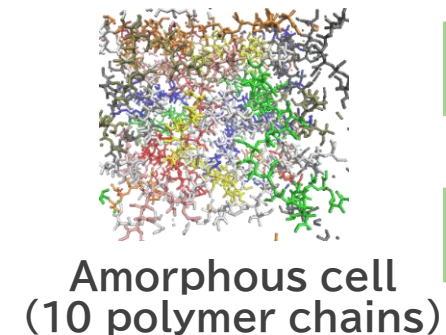
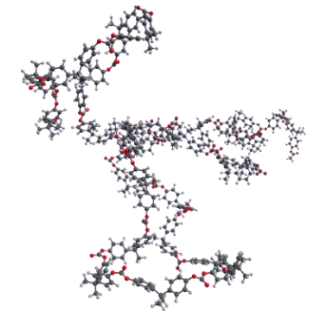
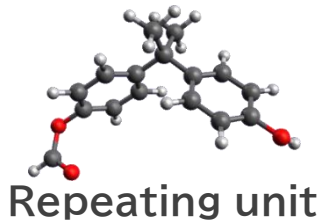
https://github.com/RadonPy/RadonPy/tree/develop/AutoMD_scripts



Example pipeline of RadonPy

Oc1ccc(cc1)C(c1ccc(cc1)OC(=O))(C)C

SMILES



Example input file

- It is assumed that these AutoMD_scripts are run from a shell script including input options.
- The input options are given by environment variables.
- Documents about the input options are available in README of AutoMD_scripts in RadonPy GitHub.

```
#!/bin/bash

### Addition of parameters of Job manager

### Loading runtime environment for LAMMPS
# module load mpi
# module load cuda
# module load lammps
# export LAMMPS_EXEC=/opt/bin/lammps/lammps

### Loading python environment
# conda activate radonpy

#####
# RadonPy settings #
#####
## Set IDs and SMILES
export RadonPy_DBID="PE_1"
export RadonPy_SMILES="*CC*"
export RadonPy_Monomer_ID="PE"
```

```
## QM calculation settings
# export RadonPy_QM_Method='wb97m-d3bj'
# export RadonPy_Charge='RESP'
# export RadonPy_FF='GAFF2_mod'
# export RadonPy_Do_TDDFT='False'
# export RadonPy_QM_TD_Method='cam-b3lyp-d3bj'

## Computational resource settings
export RadonPy_OMP_Psi4='8'
export RadonPy_MEM_Psi4='20000' # unit: MByte
# export RadonPy_Conf_MM_OMP='1'
# export RadonPy_Conf_MM_MPI='1'
# export RadonPy_Conf_MM_GPU='0'
# export RadonPy_Conf_MM_MP='0'
# export RadonPy_Conf_Psi4_OMP='4'
# export RadonPy_Conf_Psi4_MP='0'

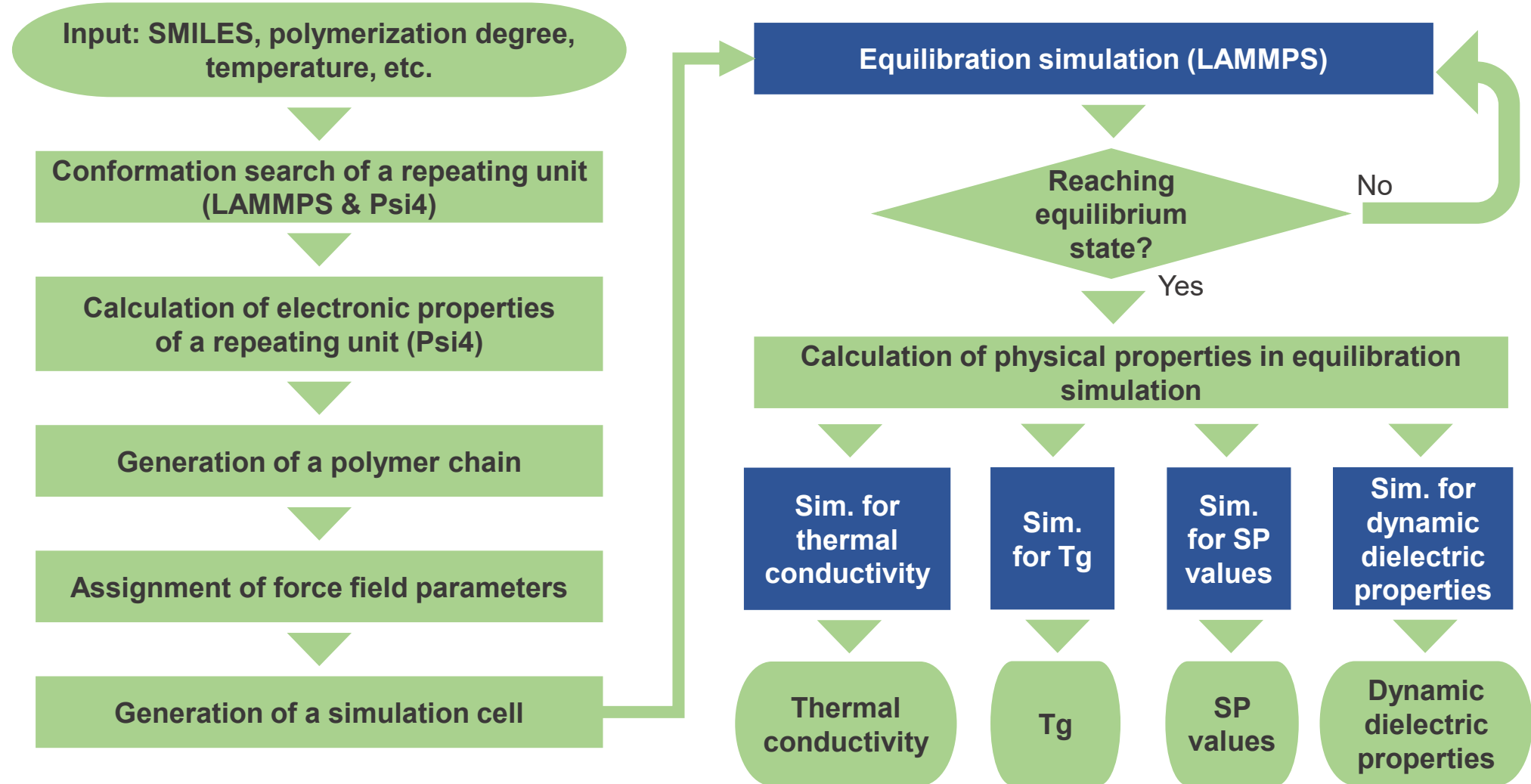
python3 /path/to/AutoMD_scripts/0_qm.py
```

Example output file

- Calculation results and meta data are outputted to `./${work_dir}/analyze/results.csv`
- Documents about the output are available in README of AutoMD_scripts in RadonPy GitHub.

DBID											
UUID	monomer_ID	ter_ID_1	ter_ID_2	smiles_list	monomer_dir	ter_dir	copoly_ratio_list	copoly_type	input_natom	...	
PI249_02	68df756c-d435-45f7-b0ae-c580a0e023db	PI249	CH3	NaN	*C(C*)C#N	NaN	NaN	1	NaN	300	...
1 rows × 180 columns											
...											
TC_ke	TC_pe	TC_pair	TC_bond	TC_angle	TC_dihed	TC_improper	TC_kspace	TC_fix	thermal_diffusivity		
...	0.005003	0.00248	0.05122	0.041765	0.069141	0.000765	0.0	0.001273	0.0	5.740983e-08	

Important functions in each step



Repeating unit generation from SMILES

radonpy.core.utils.mol_from_smiles

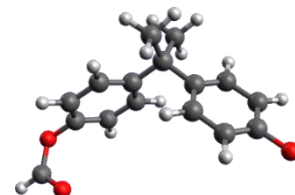
```
from radonpy.core import utils  
  
smiles = '*CC(*)c1ccccc1'  
mol = utils.mol_from_smiles(smiles)
```

SMILES

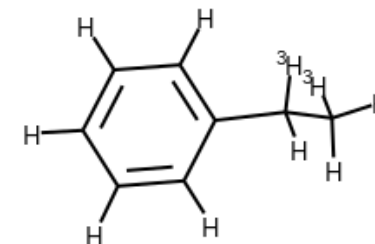
Oc1ccc(cc1)C(c1ccc(cc1)OC(=O)) (C)C



Monomer



- **Function:** For a given SMILES string of a polymer repeating unit, 3D atomic coordinates were generated using the ETKDG method.
- **Input:** a SMILES string has two asterisk symbols for representing two attachment points of the repeating unit (a.k.a PSMILES).
- **Output:** RadonPy Mol object which is extended based on RDKit Mol object
- **Remarks:**
 - ❑ The two attachment points were capped with tritium atoms which are internally treated as the attachment points of the repeating unit.
 - ❑ For unidentified *cis/trans* isomer, the *trans* isomer is generated in default.
 - ❑ For unidentified *R/S* chiral isomer, the *S* isomer is generated in default.



Example of a repeating unit of polystyrene

Mol object of RadonPy

RDKit Mol object contains information of atoms, bonds, atomic coordinates, etc.
RadonPy extends the RDKit Mol object for polymer modeling and MD calculation.

Atom object

```
Mol.GetAtoms()  
atom = Mol.GetAtomWithIdx(idx)
```

Returns a sequence containing all of the molecule's Atoms.
Returns a particular Atom.

```
atom.GetIdx()  
atom.GetAtomicNum()  
atom.GetSymbol()  
atom.GetMass()
```

Returns the atom's index
Returns the atomic number.
Returns the atomic symbol
Returns the atomic mass.

Bond object

```
Mol.GetBonds()  
bond = Mol.GetBondWithIdx(idx)
```

Returns a sequence containing all of the molecule's Bonds.
Returns a particular Bond.

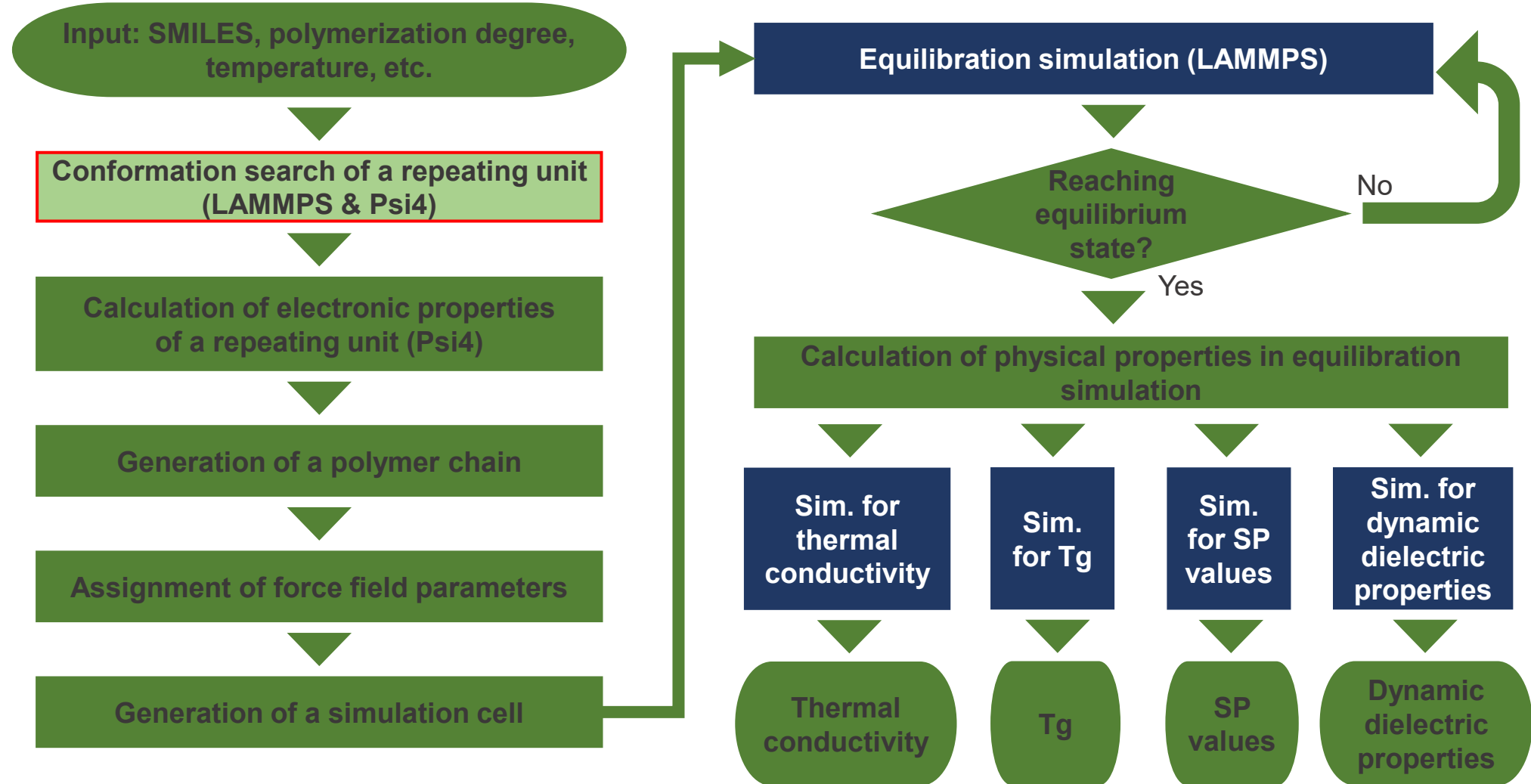
```
bond.GetIdx()  
bond.GetBondTypeAsDouble()  
bond.IsInRing()  
bond.GetBeginAtom()  
bond.GetEndAtom()
```

Returns the bond's index
Returns the type of the bond as a double (i.e. 1.0 for SINGLE, 1.5 for AROMATIC, 2.0 for DOUBLE)
Returns whether or not the bond is in a ring of any size.
Returns the bond's first atom.
Returns the bond's second atom.

Reference sites

<https://www.rdkit.org/docs/>

Important functions in each step



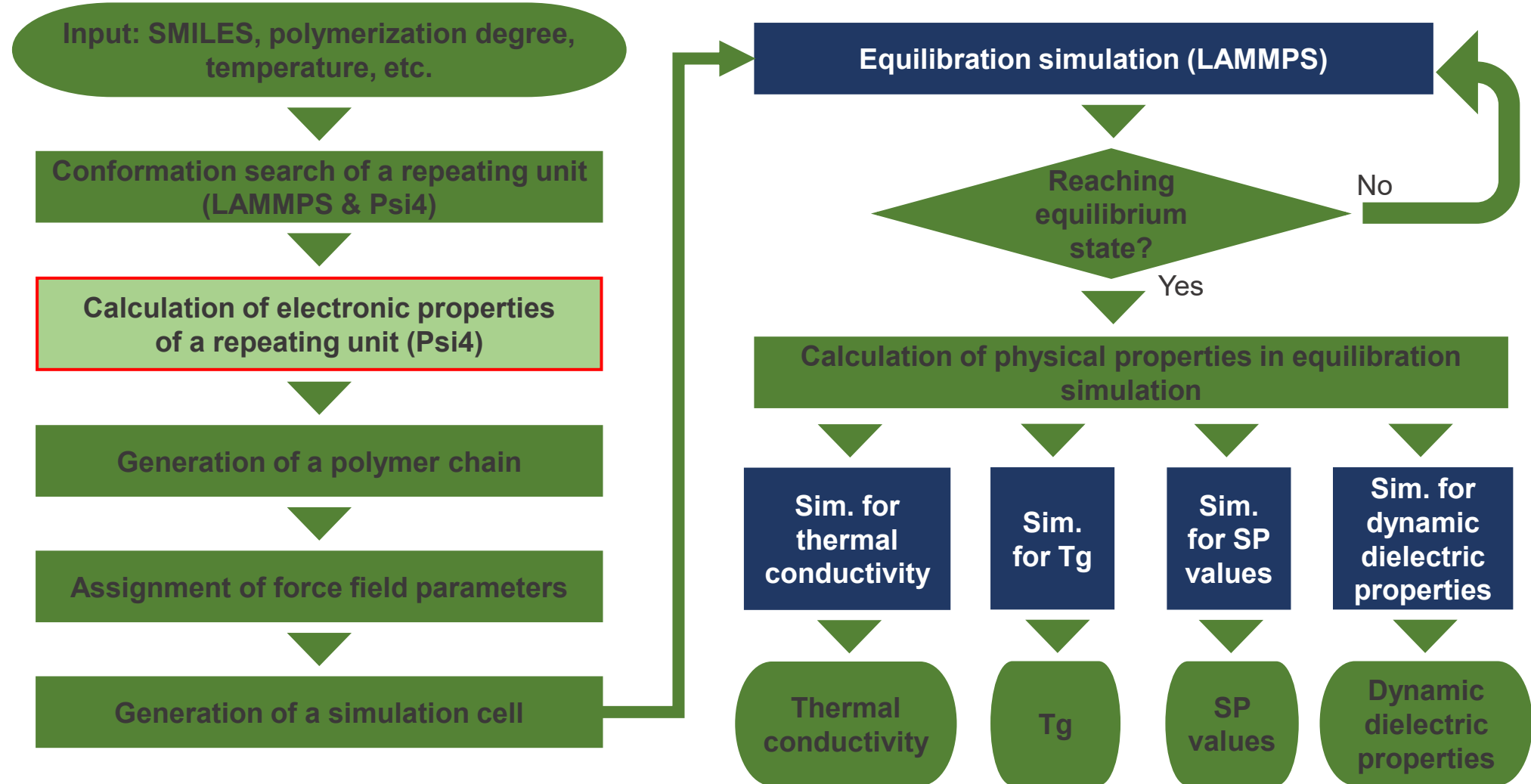
Conformation search

```
from radonpy.core import utils
from radonpy.sim import qm
from radonpy.ff.gaff2_mod import GAFF2_mod

ff = GAFF2_mod() # Instance of GAFF2_mod force field
mol = utils.mol_from_smiles(smiles) # Generation of mol object
mol, energy = qm.conformation_search(mol, ff=ff, work_dir=work_dir, psi4omp=omp_psi4, mpi=mpi, omp=omp, gpu=gpu, log_name='monomer1')
```

- **Function:** automatic search of the most stable conformer of a repeating unit
- **Input:** RadonPy Mol object of a repeating unit
- **Output:** RadonPy Mol object and total energy after geometry optimization of the most stable conformer.
- **Remarks:** The conformation search is automatically executed by following procedure
 1. **[Conformation generation]** 3D atomic coordinates of up to 1,000 different conformations were generated using the ETKDG version 2 method implemented in RDKit.
 2. **[MM optimization]** The potential energy of each conformation of a repeating unit was evaluated using the molecular mechanics calculation with GAFF2_mod after the geometry optimization.
 3. **[Clustering]** The optimized conformers were clustered by performing the Butina clustering based on the torsion fingerprint deviation.
 4. **[DFT optimization]** The most stable 4 conformations were further optimized by performing DFT calculations with the ω B97M-D3BJ/6-31G(d,p).
 5. **[Most stable conformation]** Conformation IDs in the returned Mol object are sorted by the DFT energy. The most stable conformation is set to 0 of conformation ID.

Important functions in each step



Electronic property calculation

```
from radonpy.sim import qm

qm.assign_charges(mol, charge='RESP', work_dir=work_dir, tmp_dir=tmp_dir, opt=False, omp=omp_psi4, memory=mem_psi4)
qm_data = qm.sp_prop(mol, opt=False, work_dir=work_dir, tmp_dir=tmp_dir, omp=omp_psi4, memory=mem_psi4)
polar_data = qm.polarizability(mol, opt=False, work_dir=work_dir, tmp_dir=tmp_dir, omp=conf_psi4_omp, memory=mem_psi4)
```

qm.assign_charges

Function: Calculation and assignment atomic charge of the input Mol object

Input: Mol object

Return: True (assignment success) / False (assignment fail)

qm.sp_prop

Function: Single point calculation of total energy, HOMO, LUMO, dipole moment of the input Mol Object

Input: Mol object

Return: Python dict including total energy, HOMO, LUMO, dipole moment.

qm.polarizability

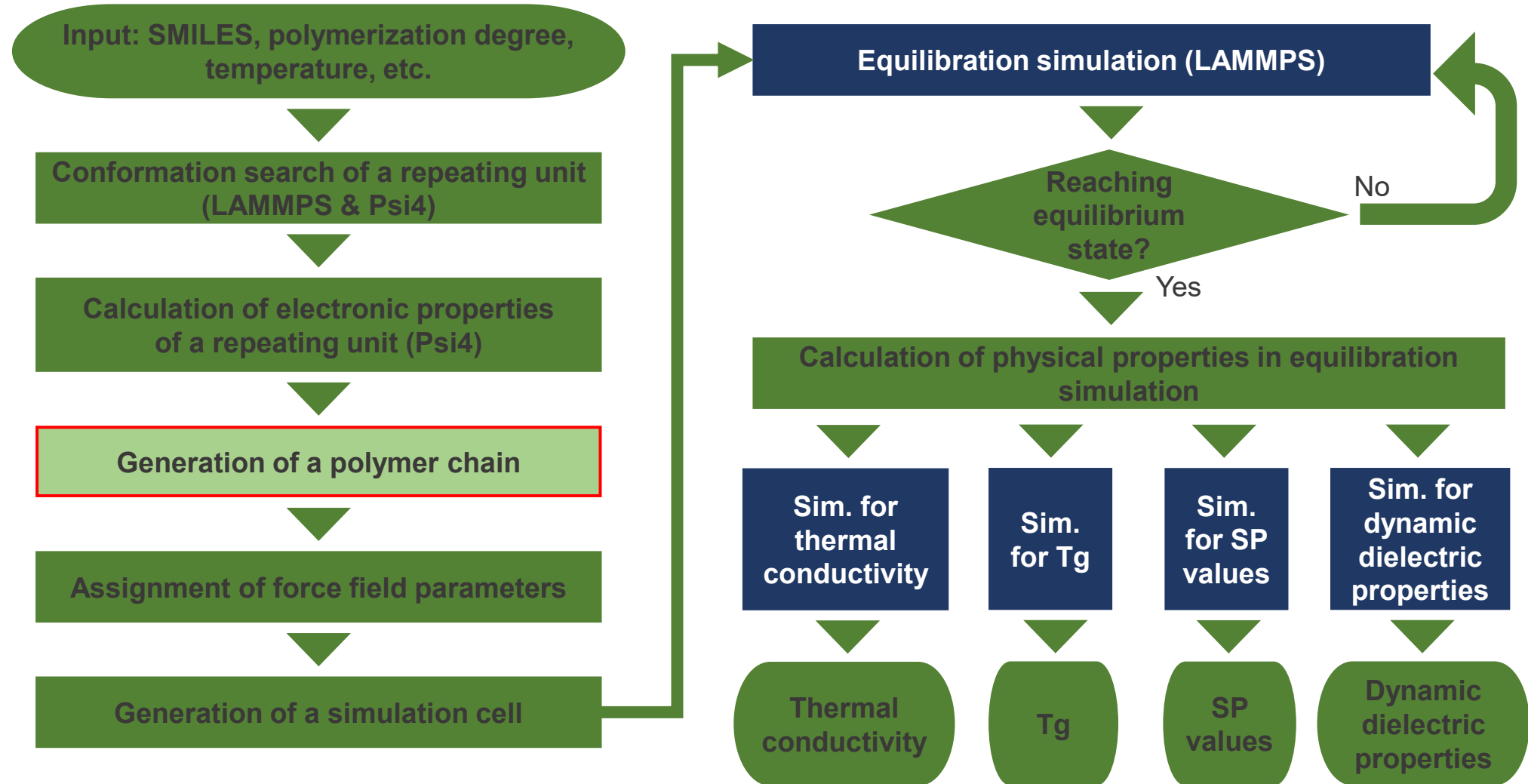
Function: Calculation of polarizability tensor of the input Mol object

Input: Mol object

Return: Python dict including polarizability and its tensor.

Notice: The electronic property calculation should be performed for a repeating unit not for polymer chain because this process requires DFT calculations.

Important functions in each step



Polymer chain generation: homopolymer

```
from radonpy.core import poly, utils

smiles = '*CC(*)c1ccccc1'
mol = utils.mol_from_smiles(smiles)
ter = utils.mol_from_smiles('*C')

n = poly.calc_n_from_num_atoms(mol, 1000, terminal1=ter)

homopoly = poly.polymerize_rw(mol, n, ter1=ter)
```

poly.polymerize_rw(mol, n)

Generation of a polymer chain with self-avoiding random walk

Inputs

mol: RadonPy Mol object of repeating unit

n: Degree of polymerization [int]

Options

ter1: RadonPy Mol object of terminating unit

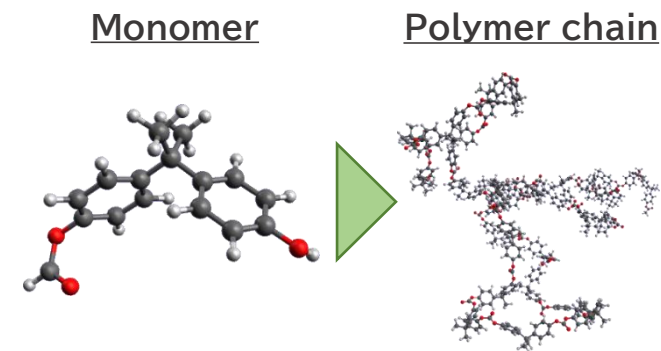
headhead: Connection type of repeating units, head-to-head (True) / head-to-tail (False) [boolean]

tacticity: Select tacticity (atactic, isotactic, or syndiotactic) [str]

atac_ratio: R/S ratio of atactic polymer [float]

poly.calc_n_from_num_atoms(mol, natom, terminal1)

Calculate the degree of polymerization to achieve the specified number of atoms (*natom*)



Polymer chain generation: copolymers

Random copolymer

```
n = poly.calc_n_from_num_atoms([mol1, mol2], 1000, ratio=[0.5, 0.5], terminal1=ter)
```

```
rcopoly = poly.random_copolymerize_rw([mol1, mol2], n, ratio=[0.5, 0.5], ter1=ter)
```

poly.random_copolymerize_rw(mols, n, ratio)

mols: List of the Mol object of repeating units

n: Degree of polymerization [int]

Options

ratio: Component ratio [list of float]



Alternating copolymer

```
acopoly = poly.copolymerize_rw([mol1, mol2], 20, ter1=ter)
```

poly.copolymerize_rw(mols, n)

mols: List of RDKit Mol object of repeating unit

n: Degree of polymerization [int]



Block copolymer

```
bcopoly = poly.block_copolymerize_rw([mol1, mol2], [20, 20], ter1=ter)
```

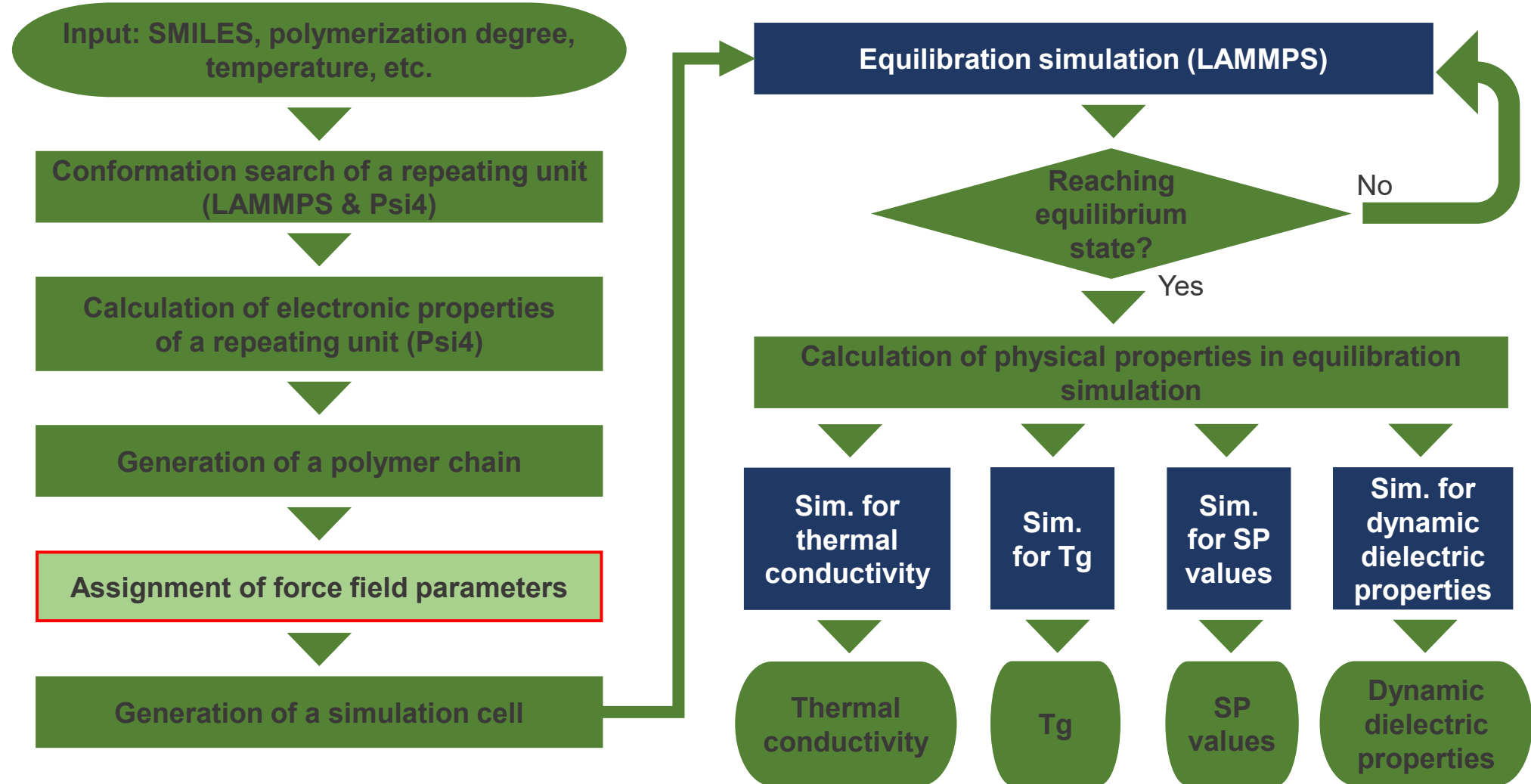
poly.block_copolymerize_rw(mols, n)

mols: List of RDKit Mol object of repeating unit

n: List of degree of polymerization [list of int]



Important functions in each step



Force field assignment

GAFF2 assignment

```
from radonpy.ff.gaff2 import GAFF2

ff = GAFF2()
result = ff.ff_assign(homopoly)
```

- Available elements : H, C, N, O, F, P, S, Cl, Br, I
- Original parameter set of GAFF2
- Return value: True (assignment success) / False (assignment fail)

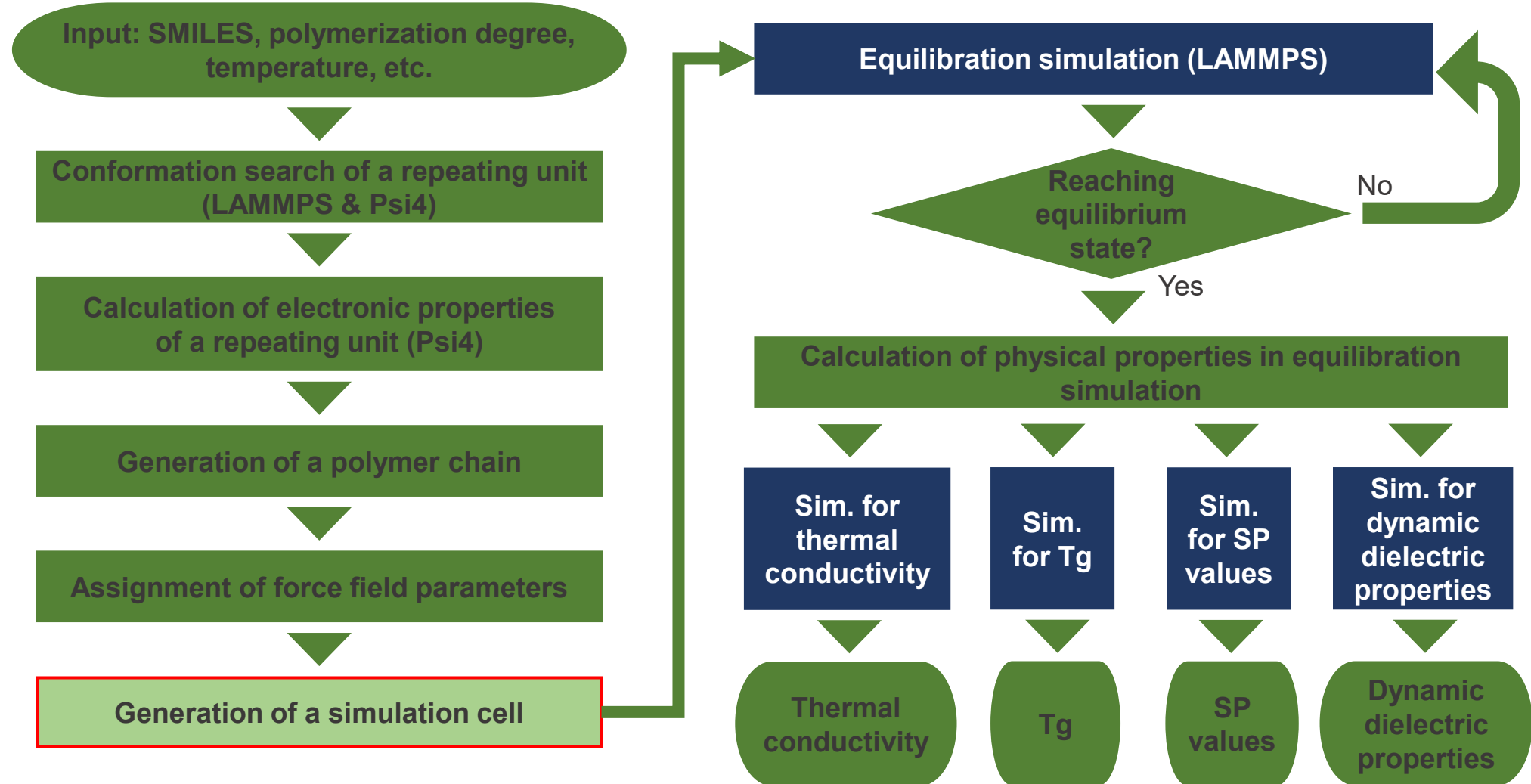
Modified GAFF2 assignment

```
from radonpy.ff.gaff2_mod import GAFF2_mod

ff = GAFF2_mod()
result = ff.ff_assign(homopoly)
```

- Available elements : H, C, N, O, F, Si, P, S, Cl, Br, I
- Modified parameters are used for fluorocarbon.
 - J. Trag, D, Zahn, *J. Mol. Model.*, **25**, 39 (2019).
- GAFF2 compatible Si parameters
 - G. I. Makarov, T. M. Makarov, *Mendeleev Communications*, **35**, 221–223 (2025).

Important functions in each step



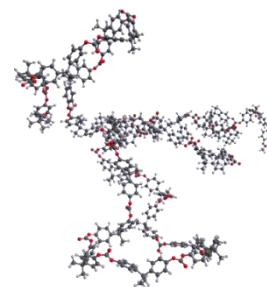
Amorphous cell generation

Generation of simulation cell of amorphous polymers (single component)

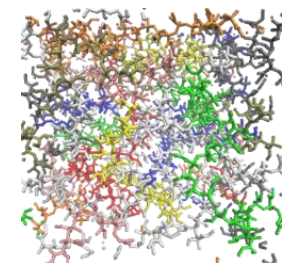
```
ac = poly.amorphous_cell(homopoly, 10, density=0.05)
```

The cell was constructed by randomly arranging and rotating 10 polymer chains such that they did not overlap with each other.

Polymer chain



Amorphous



```
def amorphous_cell(mol, n, cell=None, density=0.03, retry=10, retry_step=100, threshold=2.0, dec_rate=0.8)
```

Inputs

mol: RDKit Mol object of repeating unit
n: Number of replication of Mol object [int]

Options

cell: Adding to existing cell [Mol object]
density: Density of generating cell [float]
retry: Maximum number of retry for this function when generating unsuitable structure [int]
retry_step: Maximum number of retry for a randomly arranging step when generating unsuitable structure [int]
threshold: Threshold of minimum atom-atom distance (angstrom) [float]

Returns

RDKit Mol object

Amorphous cell generation (mixture)

Generation of simulation cell of amorphous polymers (multi component)

```
ac = poly.amorphous_mixture_cell([poly1, poly2], [5, 5], density=0.1)
```

The cell was constructed by randomly arranging and rotating five poly1 and five poly2 such that they did not overlap with each other.

```
def amorphous_mixture_cell(mols, n, cell=None, density=0.03, retry=10, retry_step=100, threshold=2.0, dec_rate=0.8)
```

Inputs

mols: List of RDKit Mol object of repeating unit
n: List of number of replication of Mol object [list of int]

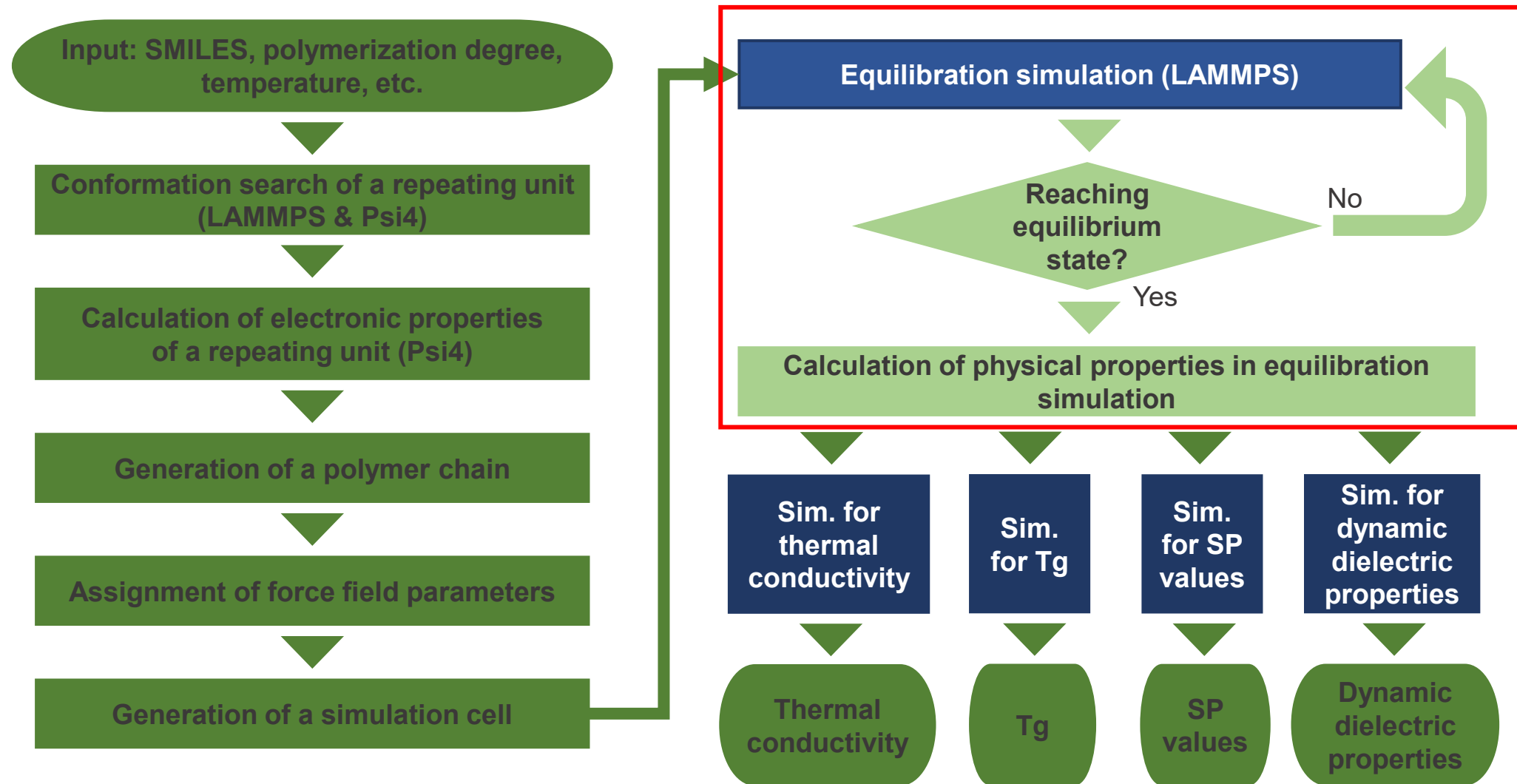
Options

cell: Adding to existing cell [Mol object]
density: Density of generating cell [float]
retry: Maximum number of retry for this function when generating unsuitable structure [int]
retry_step: Maximum number of retry for a randomly arranging step when generating unsuitable structure [int]
threshold: Threshold of minimum atom-atom distance (angstrom) [float]

Returns

RDKit Mol object

Important functions in each step



radonpy.sim.preset.eq.EQ21step

Preset of equilibration using the 21-steps compression/decompression protocol

```
from radonpy.sim.preset import eq  
  
eqmd = eq.EQ21step(ac, work_dir=work_dir)  
ac = eqmd.exec(temp=300, press=1.0, mpi=mpi, omp=omp, gpu=gpu)
```

eq.EQ21step.exec performs following 3 steps:

1. Packing simulation

Increase the density of the amorphous polymers to an appropriate value for subsequent calculations

2. The 21-steps compression/decompression protocol

[G. S. Larsen, P. Lin, K. E. Hart, and C. M. Colina, *Macromolecules* **44**, 6944–6951 (2011).]

A temperature rise to 600 K and a drop to 300 K were repeated for approximately 1.5 ns while the system was compressed to 50,000 atm and then decompressed to 1 atm by combining the NVT and NpT simulations

3. NPT equilibration

Run for 5 ns at 300 K and 1 atm

Physical property calculations from EMD

```
from radonpy.sim.preset import eq

eqmd = eq.EQ21step(ac, work_dir=work_dir)
ac = eqmd.exec(temp=300, press=1.0, mpi=mpi, omp=omp, gpu=gpu)

analy = eqmd.analyze()
prop_data = analy.get_all_prop(temp=300, press=1.0, save=True)
result = analy.check_eq()
```

eqmd.analyze()

Return an analysis object

analy.get_all_prop(temp=temp, press=press, save=True)

Calculation of physical property from equilibration MD

Return

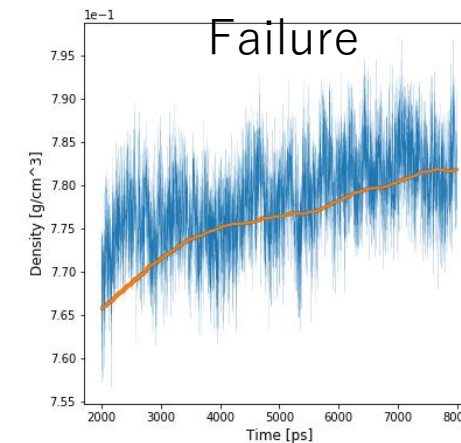
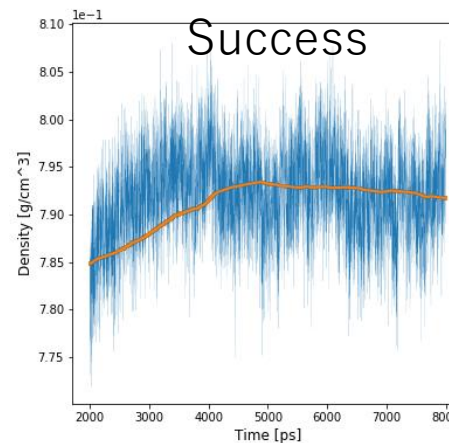
dict type (containing calculated physical properties)

analy.check_eq()

Determines the completion of the relaxation of the equilibration calculation based on the convergence status for energies, density, and Rg. This function must be executed after get_all_prop.

Return

Boolean: True (reached equilibrium state)
False (did not reach equilibrium state)



Calculations of physical properties in EMD

Specific
heat capacity

$$C_P = \frac{\langle \delta H^2 \rangle}{mk_B T^2}$$

Static dielectric
constant

$$\varepsilon = 1 + \frac{\langle \mu^2 \rangle - \langle \mu \rangle^2}{3\varepsilon_0 V k_B T^2}$$

Compressibility

$$\beta_T = \frac{\langle \delta V^2 \rangle}{V k_B T}$$

Static dielectric
constant

$$\varepsilon = n^2 + \frac{\langle \mu^2 \rangle - \langle \mu \rangle^2}{3\varepsilon_0 V k_B T^2}$$

Thermal expansion
coefficient

$$\alpha_P = \frac{\langle \delta H \cdot \delta V \rangle}{V k_B T^2}$$

Bulk modulus

$$K_T = \frac{1}{\beta_T}$$

Self-diffusion
coefficient

$$D = \frac{\langle |\mathbf{x}(t + t_0) - \mathbf{x}(t_0)|^2 \rangle}{6t}$$

Linear expansion
coefficient

$$\alpha_{lx} = \frac{\alpha_P}{3} \quad (\text{isotropic})$$

$$\alpha_{lx} = \frac{\langle \delta H \cdot \delta L_x \rangle}{L_x k_B T^2} \quad (\text{anisotropic})$$

Refractive index

$$\frac{n^2 - 1}{n^2 + 2} = \frac{4\pi}{3} \frac{\rho}{M} \alpha_{polarizability}$$

Additional EMD

Preset of additional equilibration

```
from radonpy.sim.preset import eq

eqmd = eq.Additional(ac, work_dir=work_dir)
ac = eqmd.exec(temp=300, press=1.0, mpi=mpi, omp=omp, gpu=gpu)

analy = eqmd.analyze()
prop_data = analy.get_all_prop(temp=300, press=1.0, save=True)
result = analy.check_eq()
```

eq.Additional.exec

NpT simulations were run for 5 ns at 300 K and 1 atm.
Interface is the same as EQ21step.

Sample code of EMD

```
from radonpy.sim.preset import eq

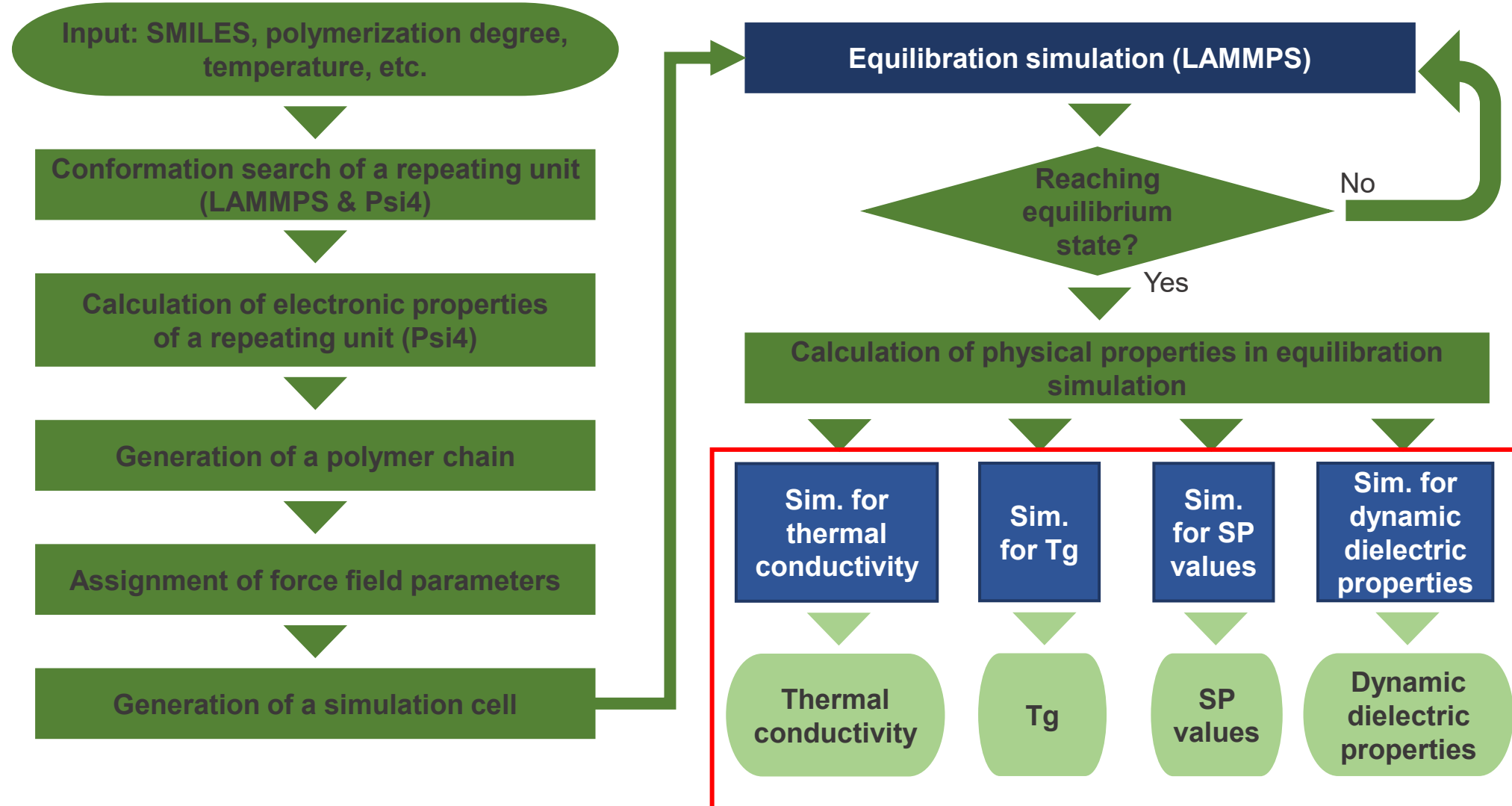
# Equilibration MD
eqmd = eq.EQ21step(ac, work_dir=work_dir)
ac = eqmd.exec(temp=temp, press=press, mpi=mpi, omp=omp, gpu=gpu)

analy = eqmd.analyze()
prop_data = analy.get_all_prop(temp=temp, press=press, save=True)
result = analy.check_eq()

# Additional equilibration MD
for i in range(4):
    if result: break
    eqmd = eq.Additional(ac, work_dir=work_dir)
    ac = eqmd.exec(temp=temp, press=press, mpi=mpi, omp=omp, gpu=gpu)
    analy = eqmd.analyze()
    prop_data = analy.get_all_prop(temp=temp, press=press, save=True)
    result = analy.check_eq()
```

- Packing simulation and the 21-steps equilibration are performed.
- After the 21-steps equilibration, NpT simulations were run for more than 5 ns at 300 K and 1 atm until equilibrium was achieved.
- The achievement of the equilibrium was checked each 5 ns after the 21-steps equilibration.

Important functions in each step



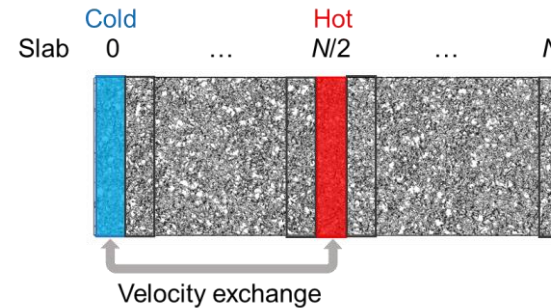
NEMD simulation of thermal conductivity

Preset of thermal conductive NEMD

```
from radonpy.sim.preset import tc

nemd = tc.NEMD_MP(mol, work_dir=work_dir, axis='x')
mol = nemd.exec(decomp=True, temp=300, mpi=mpi, omp=omp, gpu=gpu)

nemd_analy = nemd.analyze()
TC = nemd_analy.calc_tc(decomp=True, save=True)
TCdecomp = nemd_analy.TCdecomp_data
```



The heat flux is generated in the system with temperature gradients induced by exchanging the velocity between the coldest atom in slab $N/2$ and the hottest atom in slab 0.

tc.NEMD_MP(mol, work_dir=work_dir, axis='x')

Setup of thermal conductive NEMD with Müller-Plathe method.
axis: specifies the axis of heat flux direction

nemd.exec(decomp=True, temp=temp, mpi=10, omp=1, gpu=0)

Execute thermal conductive NEMD

decomp: With (True) / Without (False) decomposition analysis of thermal conductivity

nemd.analyze()

Return analysis object

nemd_analy.calc_tc(decomp=True, save=True)

Calculation of thermal conductivity.

Return

Thermal conductivity (W/mK) [float]

After `nemd_analy.calc_tc`:

`nemd_analy.TCdecomp_data` : Results of decomposition analysis [dict]

`nemd_analy.Tgrad_data['Tgrad_check']`: Result of checking the linearity of temperature gradient [boolean]

`nemd_analy.prop_df` : Results of thermal conductivity and decomposition analysis [pandas.DataFrame]

Temperature-cooling MD for Tg calculation

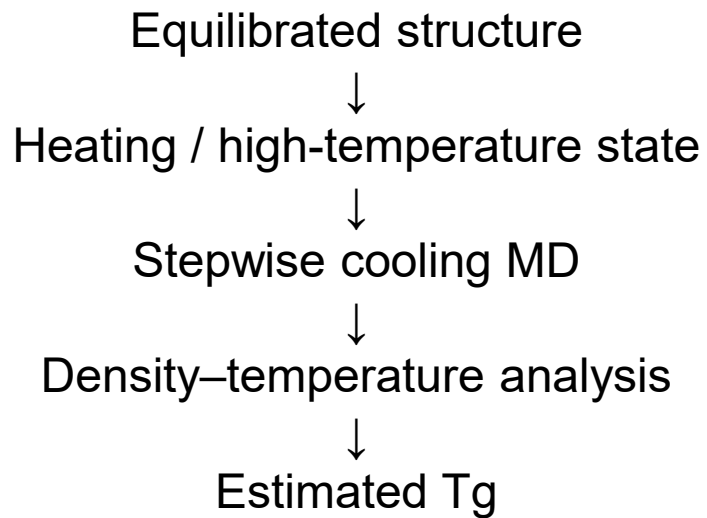
Preset of Tg calculation by temperature-cooling MD.

Tg is determined by an inflection point of density-temperature curve during cooling.

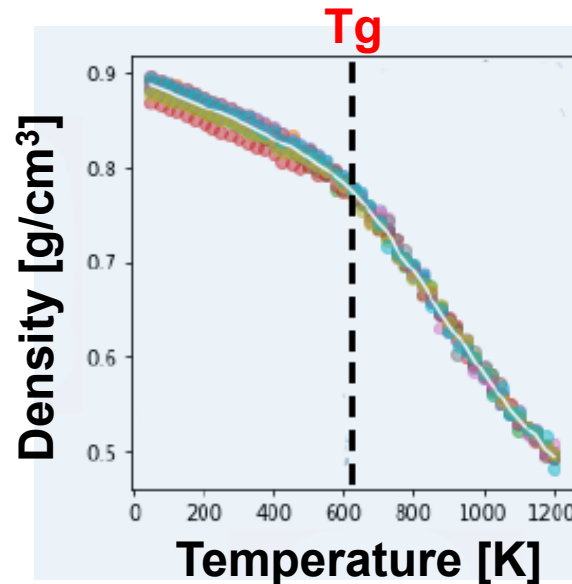
```
from radonpy.sim.preset import tg

# Tg calculation
tgmd = tg.TGMD(mol, work_dir=work_dir)
mol, tg_results = tgmd.exec(temp=temp, mpi=mpi, omp=omp, gpu=gpu, cooling_rate=8e3)
```

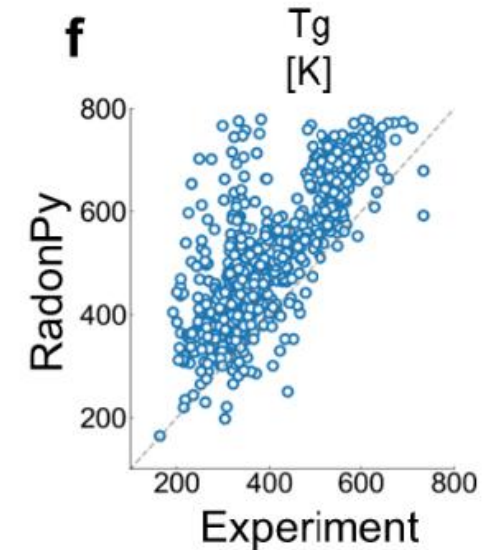
Workflow of preset.tg.TGMD



Density-temperature curve



Parity plot



Notice: In some cases, detection of the inflection point is failed leading to overestimation of Tg over 1,000 K. Therefore, we usually used a dataset of Tg less than 800 K for a training dataset of machine learnings.

Calculation of solubility parameters

- Preset of solubility parameter (SP) calculation using the trajectory of equilibrium MD.
- SP values are calculated by cohesive energy density (CED) which is estimated from energy difference between bulk (E_{bulk}) and isolated states (E_{iso}) of polymer chains.

```
from radonpy.sim.preset import sp

# SP calculation (using rerun)
spmd = sp.Rerun(mol, work_dir=work_dir)
mol, sp_results = spmd.exec(temp=temp, press=press, mpi=mpi, omp=omp, gpu=gpu)
```

$$\text{CED} = \frac{\Delta E}{V} = \frac{E_{\text{bulk}} - E_{\text{iso}}}{V} \qquad \text{SP} = \sqrt{\text{CED}}$$

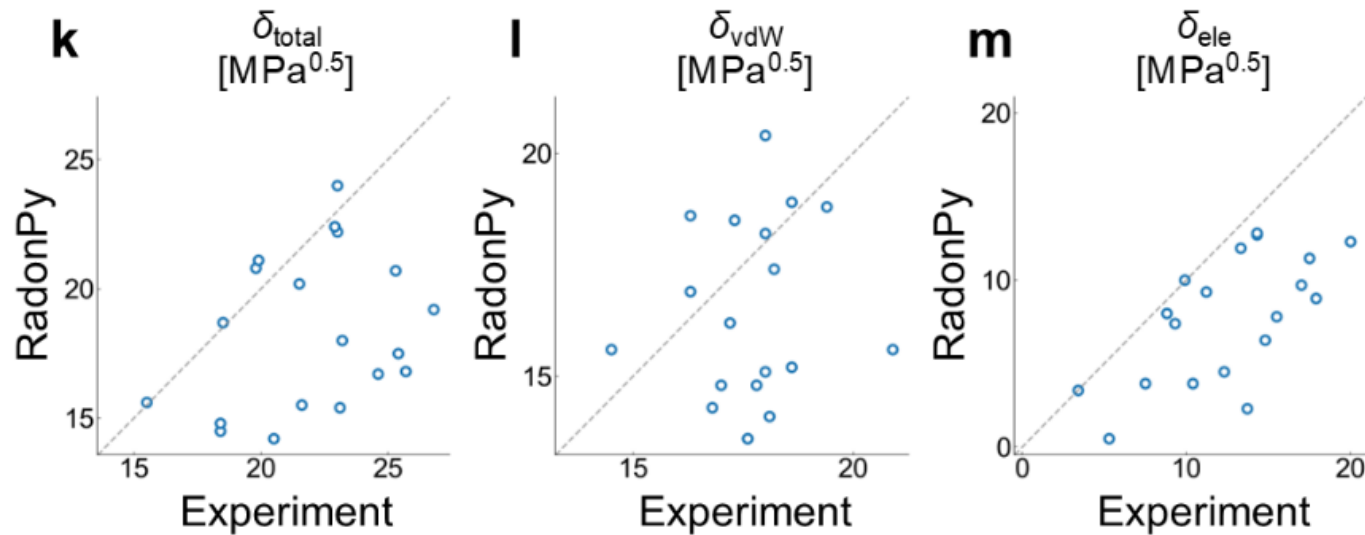
Components of the SP values (similar to Hansen SP, but not exactly the same):

The components of polar and hydrogen bonding are difficult to separate in GAFF2 force field. In this preset, the two components are summated as an electrostatic part, which is further separated to short and long components based on Coulombic pairwise energy and long-range Coulombic energy via the PPPM in LAMMPS.

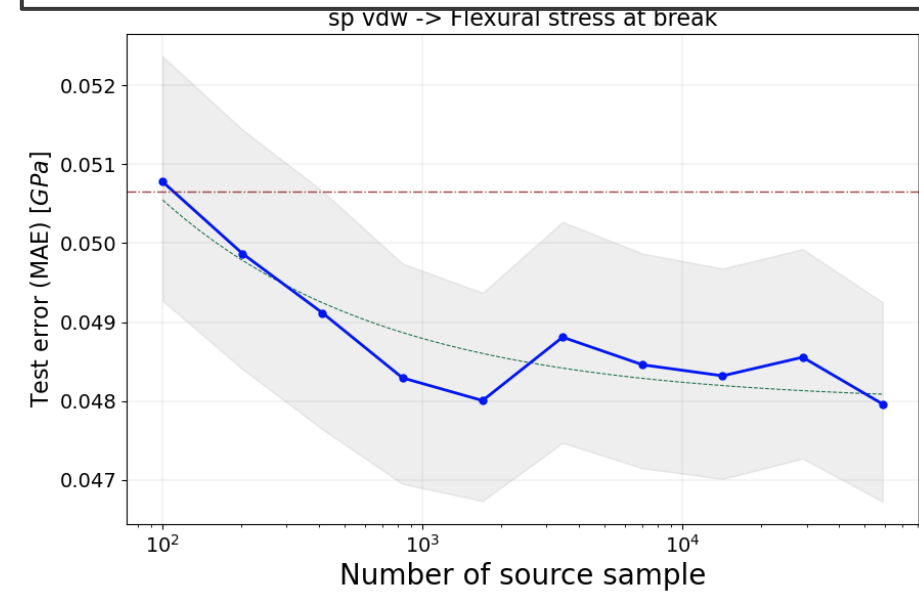
$$\text{SP}_{\text{vdW}} = \sqrt{\frac{\Delta E_{\text{vdW}}}{V}} \qquad \text{SP}_{\text{ele}} = \sqrt{\frac{\Delta E_{\text{ele}}}{V}} \qquad \text{SP}_{\text{ele-short}} = \sqrt{\frac{\Delta E_{\text{ele-short}}}{V}} \qquad \text{SP}_{\text{ele-long}} = \sqrt{\frac{\Delta E_{\text{ele-long}}}{V}}$$

Calculation of solubility parameters

Parity plots



Scaling behavior of transfer learning from δ_{vdW} to flexural stress at break



NEMD simulation of dynamic dielectric properties

Preset of dynamic dielectric properties by an alternating electric field applied NEMD.

```
#!/usr/bin/env python3

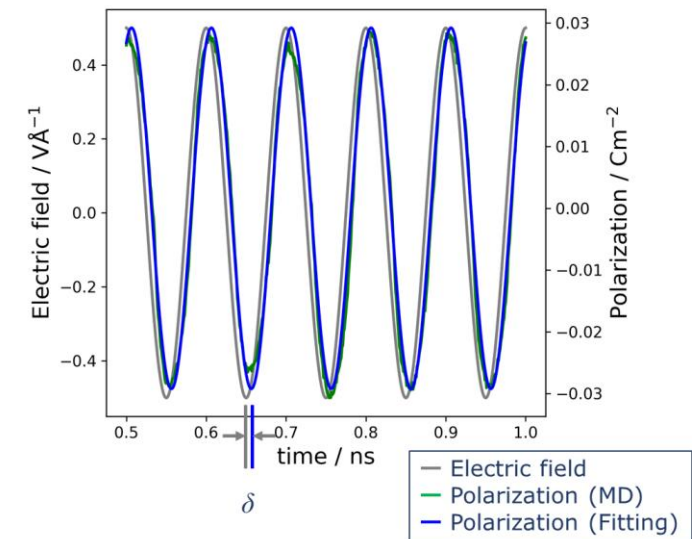
from radonpy.sim.preset import ef_dp

# Execute tuning
efeq = ef_dp.further_Additional(mol, work_dir=work_dir, axis=direction, evalue=evalue, tuning_rate=tuning_rate, process='tuning')
efeq.exec_dp(temp=temp, press=press, eq_step=tuning_step,
             mpi=mpi, omp=omp, gpu=gpu, thermo_freq=tuning_freq, dump_freq=tuning_freq)
estimated_evalue = round(efeq.tuning(), 2)

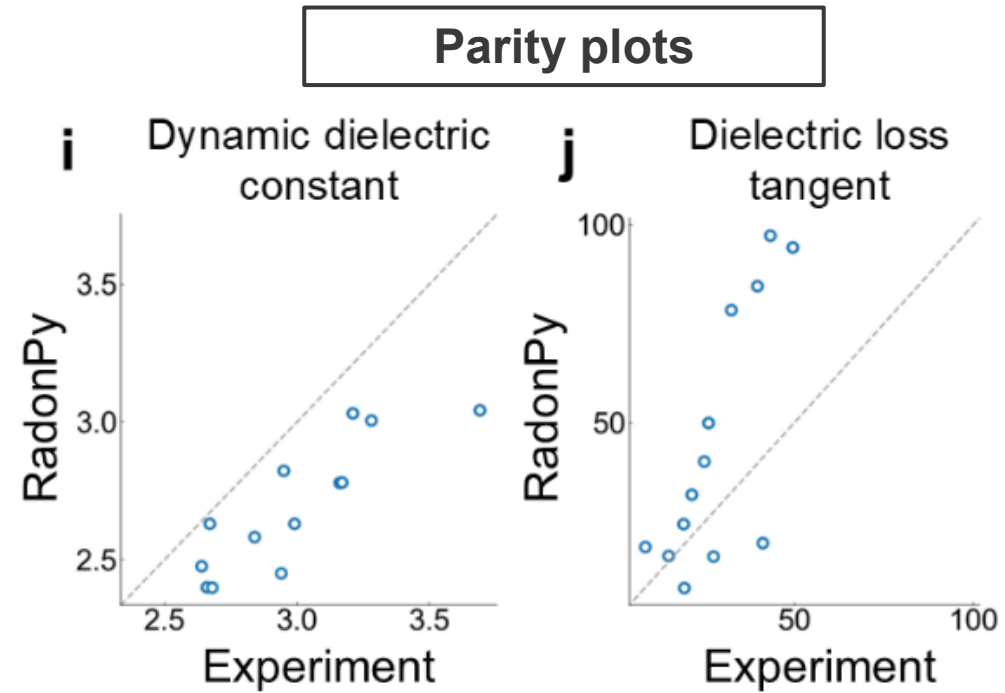
# Execute efield-MD
efeq = ef_dp.further_Additional(mol, work_dir=work_dir, axis=direction, evalue=estimated_evalue, freq=freq, process='dp')
efeq.exec_dp(temp=temp, press=press, eq_step=round_ef_step,
             mpi=mpi, omp=omp, gpu=gpu, thermo_freq=simulation_freq, dump_freq=simulation_freq)
results = eqeq.dipole_anal(props=data, printout=True, save=True)
```

This simulation is composed by following two processes:

- Tuning process: Tuning an amplitude of applying electric field in the production run.
- Production run: Applying alternating electric field and calculation of dynamic dielectric properties by phase difference between applied electric field and polarization response.



NEMD simulation of dynamic dielectric properties



Simple sample code of automated MD pipeline

```
from radonpy.core import utils, poly
from radonpy.ff.gaff2_mod import GAFF2_mod
from radonpy.sim import qm
from radonpy.sim.preset import eq, tc

smiles = '*C(C*)c1ccccc1'
ter_smiles = '*C'
temp = 300
press = 1.0
omp_psi4 = 10
mpi = 10
omp = 1
gpu = 0
mem = 10000
work_dir = './work_dir'
ff = GAFF2_mod()

if __name__ == '__main__':
    # Conformation search
    mol = utils.mol_from_smiles(smiles)
    mol, energy = qm.conformation_search(mol, ff=ff, work_dir=work_dir,
                                       psi4_omp=omp_psi4, mpi=mpi, omp=omp, memory=mem, log_name='monomer1')

    # Electronic property calculation
    qm.assign_charges(mol, charge='RESP', opt=False, work_dir=work_dir, omp=omp_psi4, memory=mem, log_name='monomer1')
    qm_data = qm.sp_prop(mol, opt=False, work_dir=work_dir, omp=omp_psi4, memory=mem, log_name='monomer1')
    polar_data = qm.polarizability(mol, opt=False, work_dir=work_dir, omp=omp_psi4, memory=mem, log_name='monomer1')

    # RESP charge calculation of a termination unit
    ter = utils.mol_from_smiles(ter_smiles)
    qm.assign_charges(ter, charge='RESP', opt=True, work_dir=work_dir, omp=omp_psi4, memory=mem, log_name='ter1')

    # Generate polymer chain
    dp = poly.calc_n_from_num_atoms(mol, 1000, terminal1=ter)
    homopoly = poly.polymerize_rw(mol, dp, ter1=ter, tacticity='atactic')
```

Simple sample code of automated MD pipeline (continued)

```
# Force field assignment
result = ff.ff_assign(homopoly)
if not result:
    print('[ERROR: Can not assign force field parameters.]')

# Generate simulation cell
ac = poly.amorphous_cell(homopoly, 10, density=0.05)

# Equilibration MD
eqmd = eq.EQ21step(ac, work_dir=work_dir)
ac = eqmd.exec(temp=temp, press=1.0, mpi=mpi, omp=omp, gpu=gpu)

analy = eqmd.analyze()
prop_data = analy.get_all_prop(temp=temp, press=1.0, save=True)
result = analy.check_eq()

# Additional equilibration MD
for i in range(4):
    if result: break
    eqmd = eq.Additional(ac, work_dir=work_dir)
    ac = eqmd.exec(temp=temp, press=press, mpi=mpi, omp=omp, gpu=gpu)
    analy = eqmd.analyze()
    prop_data = analy.get_all_prop(temp=temp, press=press, save=True)
    result = analy.check_eq()

if not result:
    print('[ERROR: Did not reach an equilibrium state.]')

# Non-equilibrium MD for thermal conductivity
else:
    nemd = tc.NEMD_MP(ac, work_dir=work_dir)
    ac = nemd.exec(decomp=True, temp=temp, mpi=mpi, omp=omp, gpu=gpu)

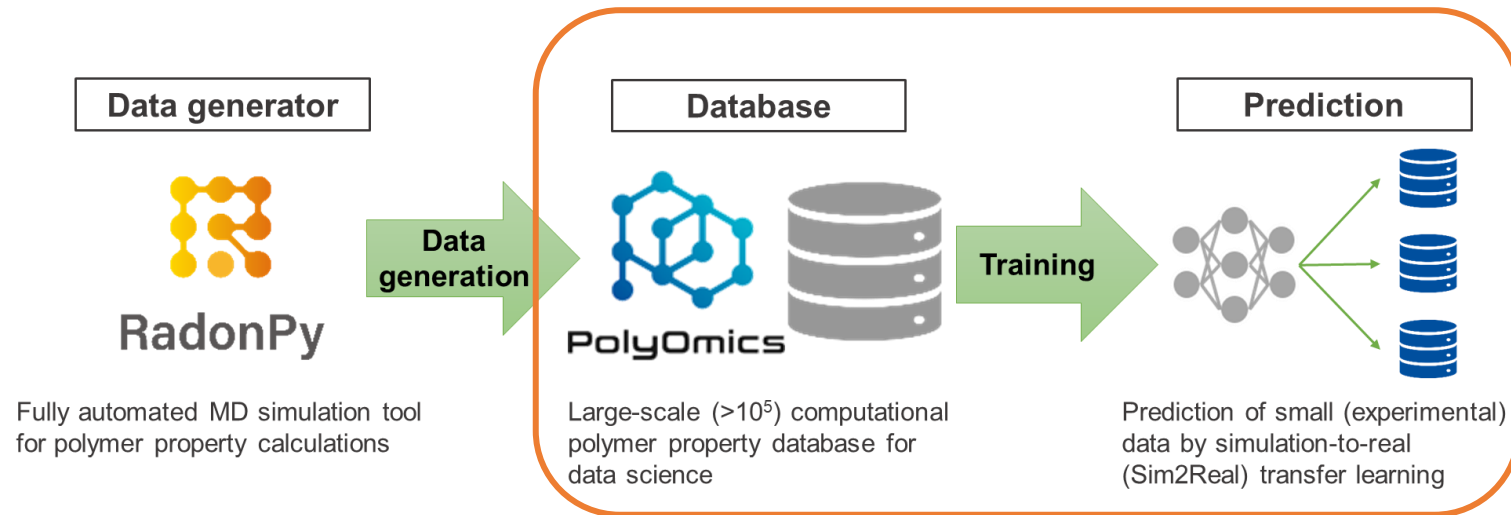
    nemd_analy = nemd.analyze()
    TC = nemd_analy.calc_tc(decomp=True, save=True)
    if not nemd_analy.Tgrad_data['Tgrad_check']:
        print('[ERROR: Low linearity of temperature gradient.]')

    print('Thermal conductivity: %f % TC')
```

Calculation results of physical properties

Summary

- RadonPy automates the full polymer MD workflow from SMILES to physical properties.
- RadonPy is a data generator in the AI era rather than a simulation tool for physical chemistry.
- Top priority of RadonPy is high-throughput, not high-accuracy.
- MD data generated by RadonPy are not perfect, but they are useful for transfer learning.



Next talk is about Sim2Real transfer learning which is bridging the gap between simulation database and real-world (experimental data) AI application
by Dr. Tomu Hamakawa

Thank you for your attention!

Are there any questions or comments?

Appendix

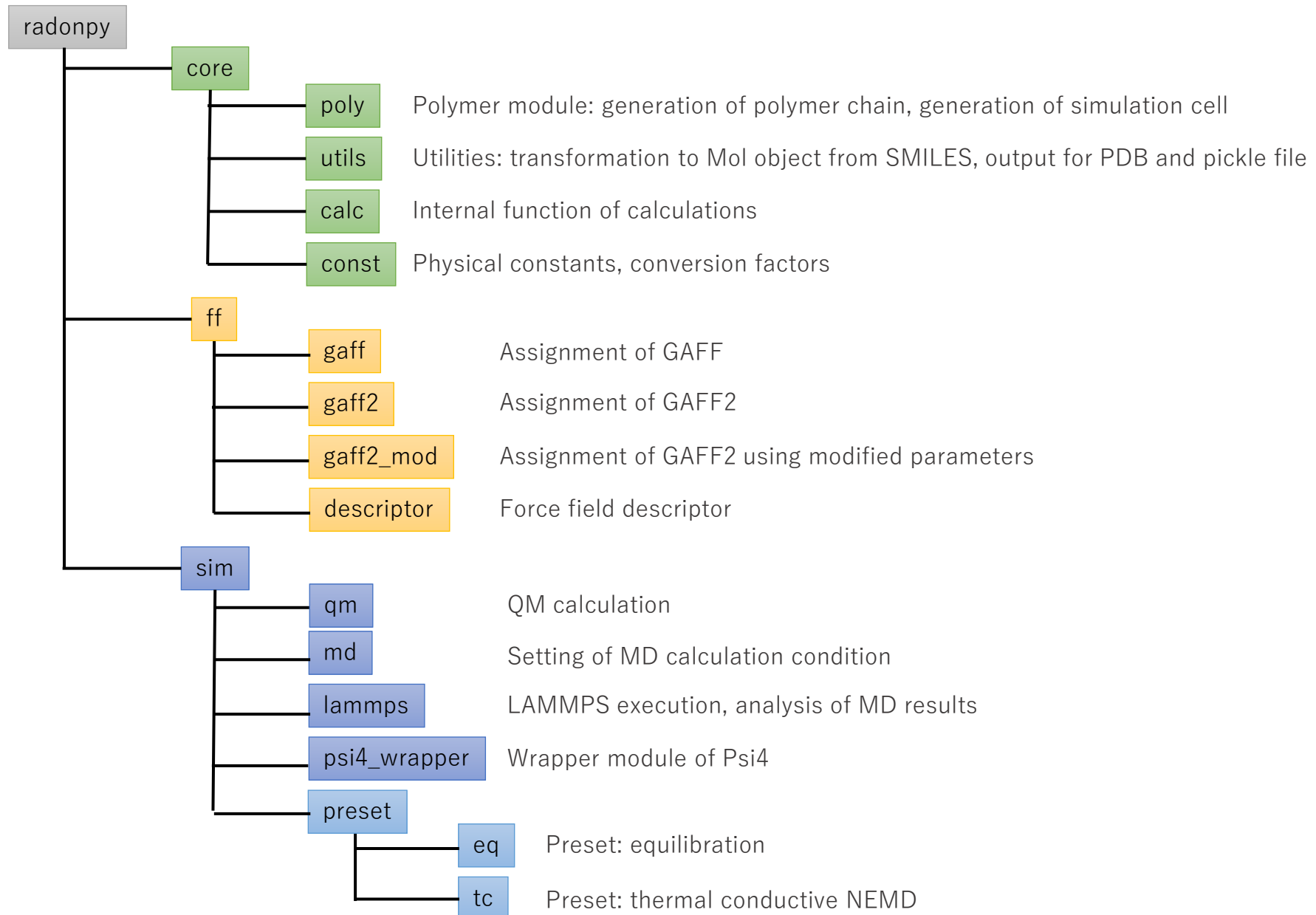
Recommended system requirement

- Python 3.7 – 3.13
- RDKit $\geq$ 2020.03
- Psi4 $\geq$ 1.5
- resp
- dftd3
- mdtraj $\geq$ 1.9
- matplotlib
- scipy
- pandas
- LAMMPS $\geq$ 3Mar20

Unit of physical property in RadonPy

Physical property	Unit
Density	g/cm^3
Radius of gyration	$\text{\AA}$
Cp, Cv	J/kg K
Compressibility	Pa^{-1}
Bulk modulus	Pa
Volume expansion coeff., Linear expansion coeff.	K^{-1}
Self-diffusion coeff.	m^2/s
Thermal conductivity	W/m K
Thermal diffusivity	m^2/s

Module structure



radonpy.sim.qm.conformation_search

```
def conformation_search(mol, ff=None, nconf=1000, dft_nconf=4, etkdg_ver=2, rmsthresh=0.5, tfdthresh=0.02, clustering='TFD',  
                        opt_method='wb97m-d3bj', opt_basis='6-31G(d,p)', opt_basis_gen={'Br': '6-31G(d,p)', 'I': 'lanl2dz'},  
                        geom_iter=50, geom_conv='QCHEM', geom_algorithm='RFO', log_name='mol', solver='lammps',  
                        solver_path=None, work_dir=None, tmp_dir=None, etkdg_omp=-1, psi4_omp=-1, psi4_mp=1,  
                        omp=1, mpi=-1, gpu=0, mm_mp=1, memory=1000, **kwargs):
```

mol: Input of RDKit Mol object

Options

ff: Force field object for MM optimization

nconf: Maximum number of generated conformations by ETKDG [int]

dft_nconf: Maximum number of conformations to be geometry optimized by DFT calculations [int]

etkdg_ver: Version of ETKDG method [int]

rmsthresh: Threshold of RMS for clustering in conformation generation with ETKDG [float]

tfdthresh: Threshold of torsion fingerprint deviation in conformation clustering [float]

opt_method: Calculation method of the DFT optimization [str]

opt_basis: Basis set of the DFT optimization [str]

geom_iter: Maximum number of iteration in the DFT optimization [int]

geom_conv: Conversion criterion in the DFT optimization [str]

geom_algorithm: Algorithm in the DFT optimization [str]

log_name: Prefix of the file name of Psi4 log in the DFT optimization [str]

work_dir: Working directory [str]

psi4_omp: Parallel number of OpenMP in DFT optimization by Psi4 [int]

omp: Parallel number of OpenMP in MM calculation by LAMMPS [int]

mpi: Process number of MPI in MM calculation by LAMMPS [int]

Returns

mol: RDKit Mol object, energy: Energy values of DFT and MM

radonpy.sim.qm.assign_charges

Calculation and assignment of atomic charge

```
def assign_charges(mol, charge='RESP', confld=0, opt=True, work_dir=None, tmp_dir=None, log_name='charge',  
                  opt_method='wb97m-d3bj', opt_basis='6-31G(d,p)', geom_iter=50, geom_conv='QCHEM', geom_algorithm='RFO',  
                  charge_method='HF', charge_basis='6-31G(d)', charge_basis_gen={'Br': '6-31G(d)', 'I': 'lanl2dz'}, **kwargs):
```

mol: Input of RDKit Mol object

Options

charge: Charge model (RESP, ESP, gasteiger, Mulliken, Lowdin) [str]

confld: Conformation ID of the RDKit mol object [int]

(confld=0 is the most stable conformation after conformation_search)

opt: With (True) /Without (False) geometry optimization by DFT calculation before the charge calculation [boolean]

work_dir: Working directory [str]

omp: Parallel number of OpenMP in Psi4 [int]

memory: Amount of memory usage (MB) in Psi4 [int]

log_name: Prefix of the file name of Psi4 log [str]

opt_method: Calculation method of the geometry optimization [str]

opt_basis: Basis set of the geometry optimization [str]

geom_iter: Maximum number of iteration in the geometry optimization [int]

geom_conv: Conversion criterion in the geometry optimization [str]

geom_algorithm: Algorithm in the geometry optimization [str]

charge_method: Calculation method of the charge calculation [str]

(The default method is HF because of the GAFF recommendation)

charge_basis: Basis set of the charge calculation [str]

charge_basis_gen: Basis set of the charge calculation for each element [dict]

Returns

Boolean: Success (True) / Fail (False)

radonpy.sim.qm.sp_prop

Calculation of total energy, HOMO level, LUMO level, and dipole moment

```
def sp_prop(mol, confId=0, opt=True, work_dir=None, tmp_dir=None, log_name='sp_prop',
            opt_method='wb97m-d3bj', opt_basis='6-31G(d,p)', opt_basis_gen={'Br': '6-31G(d,p)', 'I': 'lanl2dz'},
            geom_iter=50, geom_conv='QCHEM', geom_algorithm='RFO',
            sp_method='wb97m-d3bj', sp_basis='6-311G(d,p)', sp_basis_gen={'Br': '6-311G(d,p)', 'I': 'lanl2dz'}, **kwargs):
```

mol: Input of RDKit Mol object

Options

confId: Conformation ID of the RDKit mol object [int]
opt: With (True) /Without (False) geometry optimization by DFT calculation [boolean]
work_dir: Working directory [str]
omp: Parallel number of OpenMP in Psi4 [int]
memory: Amount of memory usage (MB) in Psi4 [int]
log_name: Prefix of the file name of Psi4 log [str]
opt_method: Calculation method of the geometry optimization [str]
opt_basis: Basis set of the geometry optimization [str]
opt_basis_gen: Basis set of the geometry optimization (for each element) [str]
geom_iter: Maximum number of iteration in the geometry optimization [int]
geom_conv: Conversion criterion in the geometry optimization [str]
geom_algorithm: Algorithm in the geometry optimization [str]
sp_method: Calculation method of the single-point calculation [str]
sp_basis: Basis set of the single-point calculation [str]
sp_basis_gen: Basis set of the geometry optimization (for each element) [dict]

Returns

dict type

qm_total_energy: Total energy (float, kJ/mol)
qm_homo: HOMO (float, eV)
qm_lumo: LUMO (float, eV)
qm_dipole_x, qm_dipole_y, qm_dipole_z: Dipole moment (float, Debye)

radonpy.sim.qm.polarizability

Calculation of polarizability tensor

```
def polarizability(mol, confId=0, opt=True, work_dir=None, tmp_dir=None, log_name='polarizability', mp=1,
                  opt_method='wb97m-d3bj', opt_basis='6-31G(d,p)', opt_basis_gen={'Br': '6-31G(d,p)', 'I': 'lanl2dz'},
                  geom_iter=50, geom_conv='QCHEM', geom_algorithm='RFO',
                  polar_method='wb97m-d3bj', polar_basis='6-311+G(2d,p)', polar_basis_gen={'Br': '6-311G(d,p)', 'I': 'lanl2dz'}, **kwargs):
```

mol: Input of RDKit Mol object

Options

confId: Conformation ID of the RDKit mol object [int]
opt: With (True) /Without (False) geometry optimization by DFT calculation [boolean]
work_dir: Working directory [str]
omp: Parallel number of OpenMP in Psi4 [int]
memory: Amount of memory usage (MB) in Psi4 [int]
log_name: Prefix of the file name of Psi4 log [str]
opt_method: Calculation method of the geometry optimization [str]
opt_basis: Basis set of the geometry optimization [str]
opt_basis_gen: Basis set of the geometry optimization (for each element) [str]
geom_iter: Maximum number of iteration in the geometry optimization [int]
geom_conv: Conversion criterion in the geometry optimization [str]
geom_algorithm: Algorithm in the geometry optimization [str]
polar_method: Calculation method of the polarizability calculation [str]
polar_basis: Basis set of the polarizability calculation [str]
polar_basis_gen: Basis set of the polarizability calculation (for each element) [dict]

Returns

dict type

qm_polarizability (float, Å³)
qm_polarizability_xx, qm_polarizability_yy, qm_polarizability_zz,
qm_polarizability_xy, qm_polarizability_xz, qm_polarizability_yz (float, Å³)

radonpy.core.poly.polymerize_rw

```
def polymerize_rw(mol, n, headhead=False, confId=0, tacticity='atactic', atac_ratio=0.5, tac_array=None,
                  retry=10, retry_step=100, dist_min=0.7, opt='rdkit', ff=None, work_dir=None, omp=1, mpi=1, gpu=0):
```

Function

Generation of a homopolymer chain by self-avoiding random walk

Inputs

mol: RDKit Mol object of repeating unit

n: Degree of polymerization [int]

Options

headhead: Connection type of repeating units, head-to-head (True) / head-to-tail (False) [boolean]

confId: Conformation ID of the RDKit mol object [int]

tacticity: Select tacticity (atactic, isotactic, or syndiotactic) [str]

atac_ratio: R/S ratio of atactic polymer [float]

tac_array: Specify the array of R/S isomer [list of boolean]

retry: Maximum number of retry for this function when generating unsuitable structure [int]

retry_step: Maximum number of retry for a random-walk step when generating unsuitable structure [int]

dist_min: Threshold of minimum atom-atom distance (angstrom) [float]

opt: Optimization method in polymer chain growing [str]

Returns

RDKit Mol object